

Parallax Propeller 2 Spin2 语言文档 2021-03-23 v35L

文件状态

Version	Date	Progress
	2020_02_06	Started document.
v34t	2020_07_15	DEBUG added, documentation up-to-date.
v34u	2020_07_19	DEBUG improved, documentation up-to-date.
v35	2020_11_18	DEBUG improved with anti-aliasing throughout, QSIN / QCOS added, documentation up-to-date.
v35e	2021_01_06	DEBUG_BAUD symbol added. Spin2 stack-locating bug fixed.
v35f	2021_01_29	DEBUG fixes. Was erring at 63 DEBUGs, now goes to 255. Was not always resetting the DEBUG.log file.
v35g	2021_02_13	DEBUG fixes. Line-clipping routine was causing floating-point exceptions and memory-access violations.
v35h	2021-02-15	- The first 16 LUT registers in the Spin2 interpreter were freed to allow for streamer 'imm-->LUT' usage. This is intended to support 1/2/4-bit video, via interrupt, within the same cog that the interpreter is running in. The inline-PASM limit went from \$134 down to \$124, in order to compensate. - A new DEBUG_WINDOWS_OFF symbol was added to inhibit any DEBUG windows from opening after a download. DEBUG_BAUD can now be set to alter the baud rate that DEBUG uses with PNut.exe.
v35i	2021-02-20	- Added command-line DEBUG-only mode for presenting flash-programmed DEBUG data and displays. - Fixed Floating-point error in SCOPE_XY.
v35j	2021-03-16	Fixed problem with DEBUG_BAUD <> 2_000_000 not working on some boards.
v35k	2021-03-19	Added DOWNLOAD_BAUD to existing DEBUG_BAUD for overriding default 2 Mbaud download and DEBUG.
v35L	2021-03-23	Added complete command-line interface to PNut.exe and included batch files for invoking PNut.exe and returning error status to STDOUT, STDERR, and ERRORLEVEL. See "Command Line options for PNut.exe".

概述

Spin2 语言的设计非常简单，但功能强大。Spin2 不会隐藏使计算机工作的底层二进制现象，而是允许您利用它进行有效的编程。Spin2 还支持汇编语言，作为内嵌序列和独立程序。

具有编程经验的人将能够在很短的时间内对Spin2有深入的了解。学习 Spin2 将会带来好处，让您能够专注于自己的想法，而无需浏览无数的类型转换和使用规则。您的大脑会很高兴保持忙碌，编译+下载+执行时间不到 1 秒。

在旋转2中：

- 没有变量类型，只有三种字大小：BYTE（8 位）、WORD（16 位）和 LONG（32 位），每种类型都支持位字段。- 所有数学运算均以 32 位执行，并且有符号和无符号运算符可供区分。- 称为对象的程序可以轻松地将其他作者编写的许多其他对象，而无需预先了解您的项目。- 对象编译为紧凑的硬件加速字节码块，触发 cog 驻留解释器代码的短序列。- 代码不区分大小写，因此您可以随意使用大写字母。- 符号名称的长度最多为 32 个字符。

In this documentation, all keywords are in UPPERCASE for clarity and anything in lowercase represents a user-defined symbolic name.

程序结构

Spin2 程序是由一个或多个对象构建的。对象是至少包含一个公共方法以及可选常量、子对象、变量、附加方法和数据的文件。对象被组装成具有子对象的内部层次结构的顶级对象。每个对象实例在运行时都会获取自己的一组由对象定义的变量，以维持其唯一的操作状态。

对象的不同部分在块内声明，所有块均以 3 字母块标识符开头。

编译器实际上可以生成 [PASM 程序](#) 或 [Spin2+PASM 程序](#)，具体取决于 .spin2 文件中存在哪些块。

Block Identifier	Block Contents	Spin2+PASM Programs	PASM-only Programs
CON	Constant declarations (CON is the initial/default block type)	Permitted	Permitted
OBJ	Child-object instantiations	Permitted	Not Allowed
VAR	Variable declarations	Permitted	Not Allowed
PUB	Public method for use by the parent object and within this object	Required	Not Allowed
PRI	Private method for use within this object	Permitted	Not Allowed
DAT	Data declarations, including PASM code	Permitted	Required

这里 [是一些最小的 Spin2 和仅限 PASM 的程序](#)。如果将它们复制并粘贴到 PNut.exe 中，则可以按 F10 来运行它们。

Minimal Spin2 Program	<pre>PUB MinimalSpin2Program() REPEAT PINWRITE(63..56, GETRND()) WAITMS(100)</pre>	<pre>'first PUB method executes 'write a random pattern to P63..P56 'wait 1/10th of a second, loop</pre>
Minimal PASM Program	<pre>DAT loop DRVRND #56 ADDPINS 7 WAITX ##clkfreq_/10 JMP #loop</pre>	<pre>'start PASM at hub \$00000 for cog \$000 'write a random pattern to P63..P56 'wait 1/10th of a second, loop</pre>

这是一个 Spin2 程序，其中包含每个块类型。

All-Block Spin2 Program	CON _clkfreq = 297_000_000	'set clock frequency
	OBJ vga : "VGA_640x480_text_80x40"	'instantiate vga object
	VAR time, i	'declare object-wide variables
	PUB go()	'this first public method executes, cog stops after
	vga.start(8)	'start vga on base pin 8
	SEND := @vga.print	'establish SEND pointer
	SEND(4, \$004040, 5, \$00FFFF)	'set light cyan on dark cyan
	time := GETCT()	'capture time
	i := @text	'print file to vga screen
	REPEAT @textend-i	
	SEND(byte[i++])	
	time := GETCT() - time	'capture time delta in clock cycles
	time := MULDIV64(time, 1_000_000, clkfreq)	'get time delta in microseconds
	SEND(12, "Time elapsed during printing was ", dec(time), " microseconds.")	'print time delta
	PRI dec(value) flag, place, digit	'private method prints decimals, three local variables
	flag~	'reset digit-printed flag
	place := 1_000_000_000	'start at the one-billion's place and work downward
	REPEAT	
	IF flag = (digit := value / place // 10) place == 1	'print a digit?
	SEND("0" + digit)	'yes
	IF LOOKDOWN(place : 1_000_000_000, 1_000_000, 1_000)	'also print a comma?
	SEND(",")	'yes
	WHILE place /= 10	'next place, done?
	DAT	
	text FILE "VGA_640x480_text_80x40.txt"	'include raw file data for printing
textend		

A breakdown of each block type follows.

CON 块

CON 块用于定义可在整个文件中使用的符号常量。

- 符号常量解析为 32 位值。- 可以使用“=”或仅在枚举列表中表达其名称来分配符号常量。- 文件中的每个块（包括 CON 块）都可以引用符号常量。- 符号常量可以通过“objectname.constantname”语法由父对象的方法引用。
- 如果存在小数点，则该值以 IEEE-754 单精度格式编码。

CON Direct Constant Assignments	CON EnableFlow = 8 DisableFlow = 4 ColorBurstFreq = 3_579_545 PWM_base = 8 PWM_pins = PWM_base ADDPINS 7 x = 5, y = -5, z = 1 HalfPi = 1.5707963268 QuarPi = HalfPi / 2.0 j = ROUND(4000.0 / QuarPi)	'single assignments' 'comma-separated assignments' 'float to integer'
CON Enumerated Constant Assignments	CON #0,a,b,c,d #1,e,f,g,h #4[2],i,j,k,l #-1[-1],m,n,p #16 q r[0] s t u[2] v w CON e0,e1,e2	'a=0, b=1, c=2, d=3 (start=0, step=1) 'e=1, f=2, g=3, h=4 (start=1, step=1) 'i=4, j=6, k=8, l=10 (start=4, step=2) 'm=-1, n=-2, p=-3 (start=-1, step=-1) 'start=16, step=1 'q=16 'r=17 ([0] is a step multiplier) 's=17 't=18 'u=19 ([2] is a step multiplier) 'v=21 'w=22 'e0=0, e1=1, e2=2 (start=0, step=1) ..enumeration is reset at each CON

OBJ 块

OBJ 块用于将子对象实例化为当前 (parent) 对象。子对象的方法可以被执行, 并且它们的常量可以在运行时被父对象引用。

- 最多可以将 32 个不同的子对象合并到一个父对象中。
- 子对象可以单独实例化，也可以在最多 255 个的数组中实例化。
- 每个父对象最多允许 1024 个子对象。

OBJ	OBJ		
Child-Object Instantiations	vga	: "VGA_Driver"	'instantiate "VGA_Driver.spin2" as "vga"
	mouse	: "USB_Mouse"	'instantiate "USB_Mouse.spin2" as "mouse"
	v[16]	: "VocalSynth"	'instantiate an array of 16 objects '..v[0] through v[15]

从父对象方法中，可以使用以下语法调用子对象方法：

```
object_name.method_name({any_parameters})
```

在父对象方法中，可以使用以下语法引用子对象常量：

对象名.常量名

VAR块

VAR 块用于声明可由对象内的所有方法使用的符号变量。

- 变量可以是长整型 (32 位)、字 (16 位) 和字节 (8 位)。
- 变量可以声明为单个变量或数组。
- 变量按照声明的顺序打包在内存中, 从长对齐地址开始。
- 变量在运行时初始化为零。
- 每个对象的前 15 个 long 变量内存都通过特殊字节码访问, 以提高效率。
- 对象的每个实例都需要一个 long, 加上其声明的 VAR 空间量, 再加上 0.3 个字节来长对齐下一个对象的变量空间。

VAR		
Variable Declarations	VAR CogNum	'The default variable size is LONG (32 bits).
	CursorMode	
	PosX	'The first 15 longs have special bytecodes for faster/smaller code.
	Posy	
	SendPtr	'So, declare your most common variables first, as longs.
	BYTE StringChr	'byte variable (8 bits)
	BYTE StringBuff[64]	'byte variable array (64 bytes)
	BYTE a,b,c[1000],d	'comma-separated declarations
WORD CurrentCycle	'word variable (16 bits)	
WORD Cycles[200]	'word variable array (200 words)	

WORD e,f[5],g,h[10]	'comma-separated declarations
LONG Value	'long variable
LONG Values[15]	'long variable array (15 longs)
LONG i[100],j,k,l	'comma-separated declarations
ALIGNW	'word-align to hub memory, advances variable pointer as necessary
ALIGNL	'long-align to hub memory, advances variable pointer as necessary
BYTE Bitmap[640*480]	'..useful for making long-aligned buffers for FIFO-wrapping

PUB 和 PRI 块

PUB 和 PRI 块用于定义公共和私有可执行 Spin2 方法。

- PUB 方法可用于父对象以及定义它们的对象。 - PRI 方法仅可用于定义它们的对象。 - 对象中的第一个 PUB 方法是当该对象作为顶级对象运行时执行的方法。 - 方法可以有 0 到 127 个输入参数，所有参数都是单长整型。 - 方法可以有 0 到 15 个输出结果，所有结果都是单长整型。 - 方法最多可以有 64KB 的局部变量，可以是字节、字和长整型（默认），无论是单变量还是数组。 - 局部变量大小覆盖（BYTE/WORD）仅适用于正在声明的变量，不适用于后续变量。 - 结果在方法入口处初始化为零，而局部变量未定义。 - 参数、结果、局部变量按照声明的顺序打包到堆栈内存中。 - 内联 PASM 代码可以通过具有相同符号名称的寄存器访问前 16 个长参数...结果...局部变量。

PUB/PRI语法如下：

PUB/PRI 方法名称({参数{,...}}) {: 结果{,...}} {1 {ALIGNW/ALIGNL} {BYTE/WORD/LONG} localvar{{count}} {,...}}

PUB / PRI Declarations (method code would go below each declaration)	Input Parameters (longs)	Output Results (longs)	Local Variables (longs, words, bytes)
PUB go()	0	0	0
PUB SetupADC(pins)	1	0	0
PUB StartTx(pin, baud) : Okay	2	1	0
PRI RotateXY(X, Y, Angle) : NewX, NewY p,q,r	3	2	3 longs
PRI Shuffle() i, j	0	0	2 longs
PRI FFT1024(DataPtr) a, b, x[1024], y[1024]	1	0	1+1+1024+1024 longs
PRI ReMix() : Length, SampleRate WORD Buff[20000], k	0	2	20000 words + 1 long
PRI StrCheck(StrPtrA, StrPtrB) : Pass i, BYTE Str[64]	2	1	1 long + 64 bytes

DAT 块

DAT块用于表达数据和PASM代码。

- 数据按照声明的顺序打包在内存中，从长对齐地址开始。 - 数据使用以下语法表示： {symbolname} BYTE/WORD/LONG data {[count]} {,data...} - 数据和 PASM 指令之前的符号解析为地址 - 在 Spin2+PASM 程序中，集线器地址相对于对象的开头，可以按如下方式引用： - 'SymbolName' 将根据其大小（字节/字/长）返回符号处的数据。 - '@SymbolName' 将返回数据的地址。 - '@@SymbolName' 会将数据中的 '@Symbol' 转换为绝对地址（请参阅“DAT 数据指针”） - 在仅限 PASM 的程序中，集线器地址是绝对的。

DAT Symbols and Data			
DAT			'symbols without data take the size of the previous declaration
HexChrs symbol0	BYTE	"0123456789ABCDEF"	'HexChrs is a byte symbol that points to "0" 'symbol0 is a byte symbol that points after "F"
Pattern symbol1	WORD	\$CCCC,\$3333,\$AAAA,\$5555	'Pattern is word symbol that points to \$CCCC 'symbol1 is a word symbol that points after \$55555
Billions symbol2	LONG	1_000_000_000	'Billions is a long symbol that points to 1_000_000_000 'symbol2 is a long symbol that points after 1_000_000_000
DoNothing symbol3	NOP		'DoNothing is a long symbol that points to a NOP instruction 'symbol3 is a long symbol that points after the NOP instruction
symbol4	BYTE		'symbol4 is a byte symbol that points to \$78
symbol5	WORD		'symbol5 is a word symbol that points to \$5678
symbol6	LONG		'symbol6 is a long symbol that points to \$12345678
	LONG	\$12345678	'long value \$12345678
	BYTE	100[64]	'64 bytes of value 100
	BYTE	10, WORD 500, LONG \$FC000	'BYTE/WORD/LONG overrides allowed for single values
	BYTE	FVAR 99, FVARS -99	'FVAR/FVARS overrides allowed, can be read via RFVAR/RFVARS
FileDat	FILE	"Filename"	'include binary file, FileDat is a byte symbol that points to file
	ALIGNW		'word-align to hub by emitting a zero byte, if necessary
	ALIGNL		'long-align to hub by emitting 1 to 3 zero bytes, if necessary

DAT Data Pointers			
DAT			
Str0	BYTE	"Monkeys",0	'strings with symbols
Str1	BYTE	"Gorillas",0	
Str2	BYTE	"Chimpanzees",0	
Str3	BYTE	"Humanzees",0	
StrList	WORD	@Str0	'in Spin2, these are offsets of strings relative to start of object
	WORD	@Str1	'in Spin2, @@StrList[i] will return address of Str0..Str3 for i = 0..3
	WORD	@Str2	'in PASM-only programs, these are absolute addresses of strings
	WORD	@Str3	'(use of WORD supposes offsets/addresses are under 64KB)

DAT Cog-exec			
DAT	ORG		'begin a cog-exec program (no symbol allowed before ORG)
IncPins	MOV	DIRA,\$FF	'COGINIT(16, @IncPins, 0) will launch this program in a free cog
Loop	ADD	OUTA,#1	'to Spin2 code, IncPins is the 'MOV' instruction (long)
	JMP	#Loop	'to Spin2 code, @IncPins is the hub address of the 'MOV' instruction
			'to PASM code, Loop is the cog address (\$001) of the 'ADD' instruction
	ORG		'set cog-exec mode, cog address = \$000, cog limit = \$1F8 (reg, both defaults)
	ORG	\$100	'set cog-exec mode, cog address = \$100, cog limit = \$1F8 (reg, default limit)
	ORG	\$120,\$140	'set cog-exec mode, cog address = \$120, cog limit = \$140 (reg)
	ORG	\$200	'set cog-exec mode, cog address = \$200, cog limit = \$400 (LUT, default limit)
	ORG	\$300,\$380	'set cog-exec mode, cog address = \$300, cog limit = \$380 (LUT)
	ADD	reg,#1	'in cog-exec mode, instructions force alignment to cog/LUT registers
	ORGF	\$040	'fill to cog address \$040 with zeros (no symbol allowed before ORGF)
	FIT	\$020	'test to make sure cog address has not exceeded \$020
x	RES	1	'reserve 1 register, advance cog address by 1, don't advance hub address
y	RES	1	'reserve 1 register, advance cog address by 1, don't advance hub address
z	RES	1	'reserve 1 register, advance cog address by 1, don't advance hub address
buff	RES	16	'reserve 16 registers, advance cog address by 16, don't advance hub address

DAT Hub-exec			
DAT	ORGH		'begin a hub-exec program (no symbol allowed before ORGH)
IncPins	MOV	DIRA,\$FF	'COGINIT(32+16, @IncPins, 0) will launch this program in a free cog
Loop	ADD	OUTA,#1	'In Spin2, IncPins is the 'MOV' instruction (long)
	JMP	#Loop	'In Spin2, @IncPins is the hub address of the 'MOV' instruction
			'In PASM, Loop is the hub address (\$00404) of the 'ADD' instruction
	ORGH		'set hub-exec mode, hub origin = \$00400, origin limit = \$100000 (both defaults)
	ORGH	\$1000	'set hub-exec mode, hub origin = \$01000, origin limit = \$100000 (default limit)
	ORGH	\$FC000,\$FC800	'set hub-exec mode, hub origin = \$FC000, origin limit = \$FC800
	FIT	\$2000	'test to make sure hub address has not exceeded \$2000

Th当涉及 hub-exec 代码时，Spin2+PASM 程序和仅 PASM 程序之间存在一些差异：

Spin2+PASM Programs	- Hub-exec code must use relative addressing, since it is not located at its place of origin. - The LOC instruction can be used to get addresses of data assets within relative hub-exec code. - ORGH must specify at least \$400, so that pure hub-exec code will be assembled. - The default ORGH address of \$400 is always appropriate, unless you are writing code which will be moved to its actual ORGH address at runtime, so that it can use absolute addressing.		
	DAT	ORGH ORGH \$FC000	'set hub-exec mode and set origin to \$400 'set hub-exec mode and set origin to \$FC000
PASM-Only Programs	- Hub-exec code may use absolute and relative addressing, since origin always matches hub address. - ORGH fills hub memory with zeros, up to the specified address.		
	DAT	ORGH ORGH \$400	'set hub-exec mode at current hub address 'set hub-exec mode and fill hub memory with zeros to \$400

Spin2语言

常数

常量解析为 32 位值，可以表示如下：

Constants	Examples	Descriptions
Decimal	1 -150 3_000_000	- Decimal values use digits '0'..'9' - Underscores '_' are allowed after the first digit for placeholder
Hexadecimal	\$1B \$AA55 \$FFFF_FFFF	- Hex values start with '\$' and use digits '0'..'9' and 'A'..'F' - Underscores '_' are allowed after the first digit for placeholder

Double Binary	%%21 %%01_23 %%3333_2222_1111_0000	- Double binary values start with '%%' and use digits '0'..'3' - Underscores '_' are allowed after the first digit for placeholdering
Binary	%0110 %1_1111_1000 %0001_0010_0011_0100	- Binary values start with '%' and use digits '0' and '1' - Underscores '_' are allowed after the first digit for placeholdering
Float	-1.0 1_250_000.0 1e9 -1.23456e-7	- Float values use digits '0'..'9' and have a '.' and/or 'e' in them - Floats are encoded in IEEE-754 single-precision 32-bit format - Underscores '_' are allowed after the first digit for placeholdering - Floats are not part of Spin2, but a library can offer floating-point functions
Character	"H"	A single character in quotes resolves to a 7-bit ASCII value

变量

在Spin2中，既有用户定义变量，也有永久变量。下面列出了用户定义的变量源，永久变量如表所示。

- VAR 变量（集线器）
- PUB/PRI 参数、返回值和局部变量（集线器）
- DAT 符号（集线器）
- Cog 寄存器

Variables (all LONG)	Variable Name	Address or Offset	Description	Useful in Spin2	Useful in Spin2-PASM	Useful in PASM-Only
Hub Locations	CLKMODE CLKFREQ	\$00040 \$00044	Clock mode value Clock frequency value	Yes Yes	Yes Yes	No No
Hub VAR	VARBASE	+0	Object base pointer, @VARBASE is VAR base, used by method-pointer calls	Maybe	No	No
Cog Registers	PR0	\$1D8	Spin2 <-> PASM communication	Yes	Yes	No
	PR1	\$1D9		Yes	Yes	No
	PR2	\$1DA		Yes	Yes	No
	PR3	\$1DB		Yes	Yes	No
	PR4	\$1DC		Yes	Yes	No
	PR5	\$1DD		Yes	Yes	No
	PR6	\$1DE		Yes	Yes	No
	PR7	\$1DF		Yes	Yes	No
	IJMP3	\$1F0	Interrupt JMP's and RET's	No	Yes	Yes
	IRET3	\$1F1		No	Yes	Yes
	IJMP2	\$1F2		No	Yes	Yes
	IRET2	\$1F3		No	Yes	Yes
	IJMP1	\$1F4		No	Yes	Yes
	IRET1	\$1F5		No	Yes	Yes
	PA	\$1F6	Pointer registers	No	Yes	Yes
	PB	\$1F7		No	Yes	Yes
	PTRA	\$1F8	Data pointer passed from COGINIT Code pointer passed from COGINIT	No	Yes	Yes
	PTRB	\$1F9		No	Yes	Yes
	DIRA	\$1FA	Output enables for P31..P0	Yes	Yes	Yes
	DIRB	\$1FB	Output enables for P63..P32	Yes	Yes	Yes
	OUTA	\$1FC	Output states for P31..P0	Yes	Yes	Yes
	OUTB	\$1FD	Output states for P63..P32	Yes	Yes	Yes
	INA	\$1FE	Input states from P31..P0	Yes	Yes	Yes
	INB	\$1FF	Input states from P63..P32	Yes	Yes	Yes

在 Spin2 中，所有变量可以作为位域进行索引和访问。此外，符号集线器变量可以覆盖 BYTE/WORD/LONG 大小ES：

Variable Usage	Example	Description
Plain	AnyVar HubVar.WORD BYTE[address] REG[register]	Hub or permanent register variable Hub variable with BYTE/WORD/LONG size override Hub BYTE/WORD/LONG by address Register, 'register' may be symbol declared in ORG section
With Index	AnyVar[index] HubVar.BYTE[index] LONG[address][index] REG[register][index]	Hub or permanent register variable with index Hub variable with size override and index Hub BYTE/WORD/LONG by address with index Register with index
With Bitfield	AnyVar.[bitfield] HubVar.LONG.[bitfield] WORD[address].[bitfield] REG[register].[bitfield]	Hub or permanent register variable with bitfield Hub variable with size override and bitfield Hub BYTE/WORD/LONG by address with bitfield Register with bitfield
With Index and Bitfield	AnyVar[index].[bitfield] HubVar.BYTE[index].[bitfield] LONG[address][index].[bitfield] REG[register][index].[bitfield]	Hub or permanent register variable with index and bitfield Hub variable with size override, index, and bitfield Hub BYTE/WORD/LONG by address with index and bitfield Register with index and bitfield

位域是一个 10 位值，其中包含ns 位 4.0 中的基位数和位 9.5 中的附加位数。位域可以用几种不同的方式定义：

Bitfield	Bit Range	Details
. [%00000_00000]	0	0 additional bits above the base bit 0, a single-bit bitfield
. [%00000_11111]	31	0 additional bits above the base bit 31, a single-bit bitfield

. [%00010_01111]	17..15	2 additional bits above the base bit 15, a three-bit bitfield
. [%11110_00000]	30..0	30 additional bits above the base bit 0, a 31-bit bitfield
. [%11111_10000]	15..0, 31..16	31 additional bits above the base bit 16, wraps around, a 32-bit bitfield
. [%00001_11111]	0, 31	1 additional bit above the base bit 31, wraps around, a 2-bit bitfield
. [23]	23	Just the base bit, adds no extra bits
. [31..20]	31..20	'Top..Bottom' syntax allowed within '[]', wraps if Top < Bottom
. [5 ADDBITS 7]	12..5	ADDBITS can be used to compute the bitfield
. [BitfieldCon]	13..9	CON BitfieldCon = 9 ADDBITS 4 'BitfieldCon useful in PASM, too
. [BitfieldVar]	?	BitfieldVar := BaseBit ADDBITS ExtraBits 'wraps if BaseBit + ExtraBits > 31

除了位域之外，还有引脚域，用于选择同一 32 引脚块（P63..P32 或 P31..P0）内的一系列 I/O 引脚。引脚字段是 11 位值，其中包含位 5..0 中的基本引脚号和位 10..6 中的附加引脚号。引脚域由与引脚接口的指令使用。

Pinfield	Pin Range	Details
PINLOW(%00000_000000)	0	0 additional pins above the base pin 0, a single-pin pinfield
PINLOW(%00000_111111)	63	0 additional pins above the base pin 63, a single-pin pinfield
PINLOW(%00011_100000)	35..32	3 additional pins above the base pin 32, a four-pin pinfield
PINLOW(%11111_001000)	7..0, 31..8	31 additional pins above the base pin 8, wraps around, a 32-pin pinfield
PINLOW(19)	19	Just the base pin, adds no extra pins
PINLOW(49..40)	49..40	'Top..Bottom' syntax allowed within '[]', wraps if Top < Bottom
PINLOW(11 ADDPINS 4)	15..11	ADDPINS can be used to compute the pinfield
PINLOW(PinfieldCon)	53..50	CON PinfieldCon = 50 ADDPINS 3 'PinfieldCon useful in PASM, too
PINLOW(PinfieldVar)	?	PinfieldVar := BasePin ADDPINS ExtraPins 'wraps if BasePin + ExtraPins > 31

表达

- 运行时表达式可以包含常量、变量和方法的返回值。
- 编译时表达式只能使用常量。
- 所有表达式都可以使用运算符。

以下是一些表达式示例:

Expression	Details
BYTE[i++]	Byte pointed to by 'i', post-increment 'i'
(digit := value / place // 10) OR place == 1	Boolean with buried 'digit' assignment
place /= 10	Divide 'place' by 10
"0" + digit	Get 'digit' character
PINREAD(17..12)	Read pins 17..12

操作员

下面是所有操作的表格 可在 Spin2 方法中使用的ators。编译时表达式可以使用一元、二元、三元和浮点运算符。

变量前缀运算符	术语（仅限方法）	优先级（术语）	分配（仅限方法）	优先级（分配）	说明	Float Exp
?? (pre) ??var 1 ??var 1 按照 XORO32 迭代 long, 返回伪随机 Var-Postfix 运算符	Term (仅方法)	Term (仅方法)	Term (仅方法)	Term (仅方法)	说明 Float Exp ++ (pre) ++var 1 ++var 1 预递增 -- (pre) --var 1 --var 1 预递减	Float Exp (post)
) ++ var++ 1 var++ 1 后置递增 (post) -- var-- 1 var-- 1 后置减量						
(post) !!	var!!	1	var!!	1	Post-logical NOT (0 → -1, non-0 → 0)	
(post) !	var!	1	var!	1	Post-bitwise NOT	
(post) \	var\x	1	var\x	1	Post-assign x	
(post) ~	var~	1	var~	1	Post-clear all bits	
(post) ~~	var~~	1	var~~	1	Post-set all bits	
Address Operators	Term (method only)	Priority (term)			Description	Float Exp
@	@symbol	1			Hub address of VAR/PUB/PRI variable or DAT symbol	
@	@method	1			Pointer to method, may be @object{[]}.method	
@@	@@x	1			Hub address of object + x, 'DAT x long @dat_symbol'	
#	#reg_symbol	1			Register address of cog/LUT DAT symbol	
Unary Operators	Term	Priority (term)	Assign (method only)	Priority (assign)	Description	Float Exp
!!, NOT	!!x	12	!= var	1	Logical NOT (0 → -1, non-0 → 0)	
!	!x	2	!= var	1	Bitwise NOT (1's complement)	
-	-x	2	= var	1	Negate (2's complement)	✓
ABS	ABS x	2	ABS= var	1	Absolute value	✓
ENCOD	ENCOD x	2	ENCOD= var	1	Encode MSB, 0..31	
DECOD	DECOD x	2	DECOD= var	1	Decode, 1 << (x & \$1F)	
BMASK	BMASK x	2	BMASK= var	1	Bitmask, (2 << (x & \$1F)) - 1	
ONES	ONES x	2	ONES= var	1	Sum all '1' bits, 0..32	
SQRT	SQRT x	2	SQRT= var	1	Square root of unsigned value	
QLOG	QLOG x	2	QLOG= var	1	Unsigned value to logarithm {5'whole, 27'fraction}	
QEXP	QEXP x	2	QEXP= var	1	Logarithm to unsigned value	
Binary Operators	Term	Priority (term)	Assign (method only)	Priority (assign)	Description	Float Exp
>>	x >> y	3	var >>= y	17	Shift x right by y bits, insert 0's	
<<	x << y	3	var <<= y	17	Shift x left by y bits, insert 0's	
SAR	x SAR y	3	var SAR= y	17	Shift x right by y bits, insert MSB's	
ROR	x ROR y	3	var ROR= y	17	Rotate x right by y bits	
ROL	x ROL y	3	var ROL= y	17	Rotate x left by y bits	
REV	x REV y	3	var REV= y	17	Reverse y LSBs of x and zero-extend	
ZEROX	x ZEROX y	3	var ZEROX= y	17	Zero-extend above bit y	
SIGNX	x SIGNX y	3	var SIGNX= y	17	Sign-extend from bit y	
&	x & y	4	var &= y	17	Bitwise AND	
^	x ^ y	5	var ^= y	17	Bitwise XOR	
	x y	6	var = y	17	Bitwise OR	
*	x * y	7	var *= y	17	Signed multiply	✓
/	x / y	7	var /= y	17	Signed divide, return quotient	✓
+/	x +/ y	7	var +/= y	17	Unsigned divide, return quotient	
//	x // 7	7	var //= y	17	Signed divide, return remainder	
+/	x +/ y	7	var +/= y	17	Unsigned divide, return remainder	
SCA	x SCA y	7	var SCA= y	17	Unsigned scale, (x * y) >> 32	
SCAS	x SCAS y	7	var SCAS= y	17	Signed scale, (x * y) >> 30	
FRAC	x FRAC y	7	var FRAC= y	17	Unsigned fraction, (x << 32) / y	
+	x + y	8	VAR += y	17	Add	✓
-	x - y	8	var -= y	17	Subtract	✓
#>	x #> y	9	var #>= y	17	Force x => y, signed	✓
<#	x <# y	9	var <#= y	17	Force x <= y, signed	✓
ADDBITS	x ADDBITS y	10	var ADDBITS= y	17	Make bitfield, (x & \$1F) (y & \$1F) << 5	
ADDPINS	x ADDPINS y	10	var ADDPINS= y	17	Make pinfield, (x & \$3F) (y & \$1F) << 6	
<	x < y	11			Signed less than (returns 0 or -1)	✓
+<	x +< y	11			Unsigned less than (returns 0 or -1)	
<=	x <= y	11			Signed less than or equal (returns 0 or -1)	✓
+<=	x +<= y	11			Unsigned less than or equal (returns 0 or -1)	

==	x == y	11			Equal (returns 0 or -1)	✓
<>	x <> y	11			Not equal (returns 0 or -1)	✓
>=	x >= y	11			Signed greater than or equal (returns 0 or -1)	✓
+>=	x +>= y	11			Unsigned greater than or equal (returns 0 or -1)	
>	x > y	11			Signed greater than (returns 0 or -1)	✓
+>	x +> y	11			Unsigned greater than (returns 0 or -1)	
<=>	x <=> y	11			Signed comparison (<,<=,> returns -1,0,1)	✓
&&, AND	x && y	13	var &&= y	17	Logical AND (x <> 0 AND y <> 0, returns 0 or -1)	
^^, XOR	x ^^ y	14	var ^^= y	17	Logical XOR (x <> 0 XOR y <> 0, returns 0 or -1)	
 , OR	x y	15	var = y	17	Logical OR (x <> 0 OR y <> 0, returns 0 or -1)	
Ternary Operator	Term	Priority (term)			Description	Float Exp
? :	x ? y : z	16			If x <> 0 then choose y, else choose z	
Assign Operator			Assign (method only)	Priority (assign)	Description	Float Exp
:=			var := x v1,v2 := x,y	17	Set var to x Set v1 to x, set v2 to y, etc. ('_' on left = ignore)	
Equate Operator			Assign (CON block only)	Priority (equate)	Description	Float Exp
=			symbol = x	17	Set symbol to x in CON block	
Float Operators	Term (constant only)				Description	Float Exp
FLOAT ()	FLOAT (x)				Convert integer x to float	✓
ROUND ()	ROUND (x)				Convert float x to rounded integer	✓
TRUNC ()	TRUNC (x)				Convert float x to truncated integer	✓

内置方法

Hub Methods	Details
HUBSET (Value)	Execute HUBSET instruction using Value
CLKSET (NewCLKMODE, NewCLKFREQ)	Safely establish new clock settings, updates CLKMODE and CLKFREQ
COGSPIN (CogNum, Method ({Pars}), StkAddr)	Start Spin2 method in a cog, returns cog's ID if used as an expression element, -1 = no cog free
COGINIT (CogNum, PASMaddr, PTRaValue)	Start PASM code in a cog, returns cog's ID if used as an expression element, -1 = no cog free
COGSTOP (CogNum)	Stop cog CogNum
COGID () : CogNum	Get this cog's ID
COGCHK (CogNum) : Running	Check if cog CogNum is running, returns -1 if running or 0 if not
LOCKNEW () : LockNum	Check out a new LOCK from inventory, LockNum = 0..15 if successful or < 0 if no LOCK available
LOCKRET (LockNum)	Return a certain LOCK to inventory
LOCKTRY (LockNum) : LockState	Try to capture a certain LOCK, LockState = -1 if successful or 0 if another cog has captured the LOCK
LOCKREL (LockNum)	Release a certain LOCK
LOCKCHK (LockNum) : LockState	Check a certain LOCK's state, LockState[31] = captured, LockState[3:0] = current or last owner cog
COGATN (CogMask)	Strobe ATN input(s) of cog(s) according to 16-bit CogMask
POLLATN () : AtnFlag	Check if this cog has received an ATN strobe, AtnFlag = -1 if ATN strobed or 0 if not strobed
WAITATN ()	Wait for this cog to receive an ATN strobe

Pin Methods	Details
PINW PINWRITE (PinField, Data)	Drive PinField pin(s) with Data
PINL PINLOW (PinField)	Drive PinField pin(s) low
PINH PINHIGH (PinField)	Drive PinField pin(s) high
PINT PINTOGGLE (PinField)	Drive and toggle PinField pin(s)
PINF PINFLOAT (PinField)	Float PinField pin(s)
PINR PINREAD (PinField) : PinStates	Read PinField pin(s)
PINSTART (PinField, Mode, Xval, Yval)	Start PinField smart pin(s): DIR=0, then WRPIN=Mode, WXPIN=Xval, WYPIN=Yval, then DIR=1
PINCLEAR (PinField)	Clear PinField smart pin(s): DIR=0, then WRPIN=0
WRPIN (PinField, Data)	Write 'mode' register(s) of PinField smart pin(s) with Data
WXPIN (PinField, Data)	Write 'X' register(s) of PinField smart pin(s) with Data
WYPIN (PinField, Data)	Write 'Y' register(s) of PinField smart pin(s) with Data
AKPIN (PinField)	Acknowledge PinField smart pin(s)
RDPIN (Pin) : Zval	Read Pin smart pin and acknowledge, Zval[31] = C flag from RDPIN, other bits are RDPIN data
RQPIN (Pin) : Zval	Read Pin smart pin without acknowledge, Zval[31] = C flag from RQPIN, other bits are RQPIN data

Timing Methods	Details
GETCT () : Count	Get 32-bit system counter
POLLCT (Tick) : Past	Check if system counter has gone past 'Tick', returns -1 if past or 0 if not past
WAITCT (Tick)	Wait for system counter to get past 'Tick'
WAITUS (Microseconds)	Wait Microseconds, uses CLKFREQ
WAITMS (Milliseconds)	Wait Milliseconds, uses CLKFREQ
GETSEC () : Seconds	Get seconds since booting, uses 64-bit system counter and CLKFREQ, rolls over every 136 years.
GETMS () : Milliseconds	Get milliseconds since booting, uses 64-bit system counter and CLKFREQ, rolls over every 49.7 days.

PASM interfacing	Details
CALL (RegOrHubAddr)	CALL PASM code at Addr, PASM code should avoid registers \$130..\$1D7 and LUT
REGEXEC (HubAddr)	Load a self-defined chunk of PASM code at HubAddr into registers and CALL it. See REGEXEC description.
REGLOAD (HubAddr)	Load a self-defined chunk of PASM code or data at HubAddr into registers. See REGLOAD description.

Math Methods	Details
ROTXY (x, y, angle32bit) : rotx, roty	Rotate (x,y) by angle32bit and return rotated (x,y)
POLXY (length, angle32bit) : x, y	Convert (length,angle32bit) to (x,y)

XYPOL(x, y) : length, angle32bit	Convert (x,y) to (length,angle32bit)
QSIN(length, angle, twopi) : y	Rotate (length,0) by (angle / twopi) * 2Pi and return y. Use 0 for twopi = \$1_0000_0000. Twopi is unsigned.
QCOS(length, angle, twopi) : x	Rotate (length,0) by (angle / twopi) * 2Pi and return x. Use 0 for twopi = \$1_0000_0000. Twopi is unsigned.
MULDIV64(mult1,mult2,divisor) : quotient	Divide the 64-bit product of 'mult1' and 'mult2' by 'divisor', return quotient (unsigned operation)
GETRND() : Rnd	Get random long (from xoroshiro128** PRNG, seeded on boot with thermal noise from ADC)

Memory Methods	Details
GETREGS(HubAddr, CogAddr, Count)	Move Count registers at CogAddr to longs at HubAddr
SETREGS(HubAddr, CogAddr, Count)	Move Count longs at HubAddr to registers at CogAddr
BYTEMOVE(Dest, Source, Count)	Move Count bytes from Source to Dest
WORDMOVE(Dest, Source, Count)	Move Count words from Source to Dest
LONGMOVE(Dest, Source, Count)	Move Count longs from Source to Dest
BYTEFILL(Dest, Value, Count)	Fill Count bytes at Dest with Value
WORDFILL(Dest, Value, Count)	Fill Count words at Dest with Value
LONGFILL(Dest, Value, Count)	Fill Count longs at Dest with Value

String Methods	Details
STRSIZE(Addr) : Size	Count bytes in zero-terminated string at Addr, return string size, not including zero terminator
STRCOMP(AddrA,AddrB) : Match	Compare zero-terminated strings at AddrA and AddrB, return -1 if match or 0 if mismatch
STRING("Text",9) : StringAddress	Compose a zero-terminated string (quoted characters and values 1..255 allowed), return address of string

Index ⇄ Value Methods	Details
LOOKUP(Index: v1, v2..v3, etc) : Value	Lookup value (values and ranges allowed) using 1-based index, return value (0 if index out of range)
LOOKUPZ(Index: v1, v2..v3, etc) : Value	Lookup value (values and ranges allowed) using 0-based index, return value (0 if index out of range)
LOOKDOWN(Value: v1, v2..v3, etc) : Index	Determine 1-based index of matching value (values and ranges allowed), return index (0 if no match)
LOOKDOWNZ(Value: v1, v2..v3, etc) : Index	Determine 0-based index of matching value (values and ranges allowed), return index (0 if no match)

使用方法

返回单个结果的方法可以用作表达式中的项：

```

x := GETRND() +/- 100 '获取 0 到 99 之间的随机数

BYTEMOVE(ToStr, FromStr, STRSIZE(FromStr) + 1)

```

返回多个结果的方法（如 POLXY）可用于向其他方法提供多个参数：

```

x,y := SumPoints(POLXY(rho1,theta1), POLXY(rho2,theta2))

...其中... SumPoints(x1, y1, x2, y2) : x, y RET
URN x1+x2, y1+y2

```

多个方法结果可以分配给变量或通过使用下划线代替变量名称来忽略：

```

x,y := ROTXY(xin,yin,theta) '同时使用 x 和 y 结果
_y := ROTXY(xin,yin,theta) '仅使用 y 结果
x_ := ROTXY(xin,yin,theta) '仅使用 x 结果

```

User-defined methods which return one or more results can also be used as instructions, where the return values are ignored. However, built-in methods such as STRSIZE, which return results, must be used as expression terms.

中止

Spin2 有一个“中止”机制，可以从任何深度的嵌套方法调用立即返回到在方法名称之前使用“\”的基本调用者。单个返回值可以从中止点传送回基本调用者：

PRI Sub1() : 错误 'Sub1 使用 ABORT 陷阱调用 Sub2 错误 := \Sub2() \ 表示调用方法并捕获任何 ABORT \Sub2() '在这种情况下，ABORT 值将被忽略 PRI Sub2() 'Sub2 调用 Sub3 Sub3() '由于 ABORT PINHIGH(0)，Sub3 永远不会返回这里 'PINHIGH 永远不会执行 PRI Sub3() 'Sub3 ABORT，返回到 Sub1，错误代码为 ABORT ErrorCode 'ABORT 并返回错误代码 PINLOW(0) 'PINLOW 从不执行

无论特定的我有多少返回值 可能会有这样的情况，当使用前面的“\”调用该方法时，将只有一个返回值，可以忽略该返回值。

如果 ABORT 后未指定任何值，则将返回零。

如果调用方法时前面带有“\”，但没有发生 ABORT，则将返回零。

如果 ABORT 在调用链中某处没有“\”陷阱的情况下执行，则 cog 返回顶层方法并执行 COGSTOP(COGID)，从而关闭自身。

The abort mechanism is intended as a means to return from a deeply nested subroutine where some error situation has developed, but it can be used for any purpose. Basically, it's a way to return to a base caller without having to check for a condition to do so at every level of the call chain. It returns all the way back to the caller with the "\" abort trap, carrying the ABORT value. You can compose hierarchical levels of "\" abort traps and ABORT points.

方法指针

方法指针是指向方法的 LONG 值，然后用于间接调用该方法。

要建立方法指针，可以在方法名称前使用 “@” 分配一个长变量。请注意，方法名称后面没有括号：

长变量: = @SomeMethod
LongVar := @SomeObject.SomeMethod
LongVar := @SomeObject[index].SomeMethod

'当前对象中的方法
'子对象中的方法
'索引子对象中的方法

方法指针可以即时生成并作为参数传递：

SetUpIO(@InMethod,@OutMethod)

然后通过以下方式使用方法指针来调用方法：

长变量()
LongVar(Par1, Par2)
变量: = LongVar():1
Var1,Var2 := LongVar(Par1):2
Var1,Var2 := POLXY(LongVar(Par1,Par2,Par3):2)

'无参数，无返回值
'两个参数，无返回值
'无参数，只有一个返回值
'一个参数两个返回值
'三个参数两个返回值

编译时不知道所指向的方法实际有多少个参数。您需要对方法指针的使用进行编码，以便提供正确数量的参数并在 “:” 后指定正确数量的返回值，以便与所指向的方法保持一致。

方法指针可以通过对象层次结构传递，以便能够从任何地方直接调用任何方法。它们还可以用于动态指向具有相同数量的参数和返回值的不同方法。

方法指针如何工作

@method 表达式生成一个 32 位值，该值具有两个位字段：

[31..20] = 方法的索引，相对于方法的对象基数。第一个方法的索引将是实例化对象数量的两倍

[19..0] = 方法的 VAR 基址。该方法的 VAR 基址又包含该方法的对象基址的地址。

通过将方法的索引和 VAR 基址放在一起
在单个 long 中建立的，可以将其视为任何 o

然后将其转换为 32 位值，并使 VAR 基址包含方法的对象基址，即完整的方法指针
还有变量。

是

为了容纳方法指针，每个对象实例为其 VAR 空间的第一个 long 保留用于对象基址。当执行 @method 表达式时，第一个 long 会写入对象的基址。

发送

SEND 是一个特殊的方法指针，它继承自调用方法，并依次传递给所有被调用的方法。其目的是为数据提供高效的输出机制。发送

可以像方法指针一样进行赋值，但它必须指向一个带有一个参数并且没有返回值的方法：

发送: = @@OutMethod

当用作方法时，SEND 会将所有参数（包括被调用方法的任何返回值）传递给 SEND 指向的方法：

SEND("你好! ", GetDigit()+*0", 13)

在 SEND 参数内调用的任何方法都将继承 SEND 指针，以便它们也可以执行 SEND 方法：

PUB Go() 发送 := @SetLED REPEAT SEND(Flash(),\$01,\$02,\$04,\$08,\$10,\$20,\$40,\$80)

PRI Flash() : x REPEAT 2 S
END(\$00,\$FF,\$00) 返回 \$A
A

PRI SetLED(x) PINWRITE(56 ADD
PINS 7, !x) 等待(125)

在上面的示例中，以下值按重复顺序输出：\$00、\$FF、\$00、\$00、\$FF、\$00、\$AA、\$01、\$02、\$04、\$08、\$10、\$20、\$40、\$80（但对于 LED 则相反）

尽管被调用的方法继承了当前的 SEND 指针，但它可能会出于自己的目的而更改它。从该方法返回后，SEND 指针将返回到调用该方法之前的状态。因此，SEND 指针值在方法调用中传播，但不在方法返回中传播。

RECV

RECV 与 SEND 一样，是一个特殊的方法指针，它继承自调用方法，并依次传递给所有被调用的方法。其目的是为数据提供高效的输入机制。

RECV 可以像方法指针一样赋值，但它必须指向一个不带参数并返回单个值的方法：

RECV: = @InMethod

使用 RECV 的示例：

```
VAR i

PUB Go() RECV := @GetPattern REPEAT PI
NWRITE(56 ADDPINS 7, !RECV()) WAITM
S(125)
```

PRI GetPattern()：模式返回解码(
i++ & 7)

在上面的示例中，以下值按重复顺序输出：\$01、\$02、\$04、\$08、\$10、\$20、\$40、\$80（但对于 LED 则相反）

尽管被调用的方法继承了当前的 RECV 指针，但它可能会出于自己的目的而更改它。从该方法返回后，RECV 指针将返回到调用该方法之前的状态。因此，RECV 指针值在方法调用中传播，但不在方法返回中传播。

流量控制

Spin2 具有三个基本的流量控制结构：

- IF / IFNOT + ELSEIF / ELSEIFNOT + ELSE - 具有随机决策逻辑的条件执行
- 案例/案例_快速 - 单目标和多个匹配测试的条件执行
- 重复 - 具有多种模式的循环执行

所有这些构造都使用相对缩进来确定哪些代码受其控制：

```
IF cog 'if cog <> 0 COGSTOP(cog-1) '..然后停止 cog PINCLEAR(av_base_pin_
ADD
PINS 4) '..然后清除引脚模式
```

流控制结构可以按任意顺序嵌套：

```
CASE 标志 0: CASE_FAST chr 0: BYTEFILL(@screen, " ", scree
n_size) col := row := 0 1: col := row := 0 2..7: flag := chr RETURN
8: IF col col-- 9: REPEAT out(" ") WHILE col & 7 10: RETURN
11: 颜色: = $00 12: 颜色: = $80 13: 换行符 ( ) 其他: out
(chr)
```

```
2: col := chr // 列 3: row := chr // 第 4..7 行: backgro
und0_[flag-$04] := chr << 8 flag := 0
```

IF / IFNOT + ELSEIF / ELSEIFNOT + ELSE

IF 构建体以 IF 或 IFNOT 开始，并可选地使用 ELSEIF、ELSEIFNOT 和 ELSE。为了成为同一决策树的一部分，这些关键字必须具有相同的缩进级别。

如果 <condition> 不为零，则执行 IF 或 ELSEIF 下的缩进代码。如果 <condition> 为零，则执行 IFNOT 或 ELSEIFNOT 下的代码。如果没有执行其他缩进代码，则执行 ELSE 下的代码：

- IF / IFNOT <条件> - 初始 IF 或 IFNOT
- <缩进代码> ELSEIF / ELSEIFNOT
- <条件> - 可选的 ELSEIF 或 ELSEIFNOT
- <缩进代码>
- 别的 - 可选的最后 ELSE

<indented code>

案例/案例_快速

案例公司 nstruct 按顺序将目标值与可能匹配的列表进行比较。当找到匹配项时，执行相关代码 es。

匹配值/范围必须缩进到 CASE 关键字之后。多个匹配值/范围可以用逗号分隔符表示。与匹配值/范围相关的任何其他代码行都必须缩进到匹配值/范围之后：

CASE target	- CASE with target value
<match> : <code>	- match value and code
<indented code>	
<match..match> : <code>	- match range and code
<indented code>	
<match>,<match..match> : <code>	- match value, range, and code
<indented code>	
OTHER : <code>	- optional OTHER case, in case no match found
<indented code>	

CASE_FAST 与 CASE 类似，但它不是按顺序将目标与可能匹配的列表进行比较，而是使用最多 256 个条目的索引跳转表来立即分支到适当的代码，从而节省时间，但可能会花费更大的编译代码。如果只有连续的匹配值而没有匹配范围，则生成的代码实际上将比具有多个匹配值的正常 CASE 构造小。

对于要编译的 CASE_FAST，匹配值/范围必须是唯一常量，且彼此之间的差值均在 255 以内。

请参阅上面“流量控制”下的 CASE_FAST 示例。

重复

所有循环都是通过 REPEAT 结构实现的，它有多种形式：

REPEAT	- Repeat forever (useful for putting at end of program if you don't want the cog to stop and cease driving its I/O's)
<indented code>	
REPEAT <count>	- Repeat <count> times, if <count> is zero then <indented code> is skipped
<indented code>	
REPEAT <variable> FROM <first> TO <last>	- Repeat while iterating <variable> from <first> to <last>, stepping by +/-1
<indented code>	- After completion, <variable> = <last> +/-1
REPEAT <variable> FROM <first> TO <last> STEP <delta>	- Repeat while iterating <variable> from <first> to <last>, stepping by +/-<delta>
<indented code>	- After completion, <variable> = <last> +/-<delta>
REPEAT WHILE <condition>	- Repeat while <condition> is not zero, <condition> is evaluated before <indented code> executes
<indented code>	
REPEAT UNTIL <condition>	- Repeat until <condition> is not zero, <condition> is evaluated before <indented code> executes
<indented code>	
REPEAT	- Repeat while <condition> is not zero, <condition> is evaluated after <indented code> executes
<indented code>	
WHILE <condition>	- WHILE must have same indentation as REPEAT
REPEAT	- Repeat until <condition> is not zero, <condition> is evaluated after <indented code> executes
<indented code>	
UNTIL <condition>	- UNTIL must have same indentation as REPEAT

在 REPEAT 结构中，有两个特殊指令可用于更改执行过程：NEXT 和 QUIT。NEXT 将立即分支到 REPEAT 构造中做出再次循环决定的点，而 QUIT 将退出 REPEAT 构造并在其后继续。这些指令通常有条件地使用：

重复

<缩进代码> IF <条件> - 可以选择强制执行 REPEAT NEXT 的下一迭代

<缩进代码> IF <条件>
- 可选择退出重复
退出<缩进代码>

内联 PASM 代码

Spin2方法 可以通过在 PASM 代码前面加上“ORG {000..\$12F}”并用 END 终止来执行内联 PASM 代码。

```
PUB go() | x

REPEAT

  ORG
    GETRND WC      'rotate a random bit into x
    RCL     x,#1
  END

  PINWRITE(56 ADDPINS 7, x) 'output x to the P2 Eval board's LEDs
  WAITMS(100)
```

您的 PASM 代码将被汇编 在末尾添加 RET 指令以确保它返回到 Spin2，以防没有早期 _RET_ 或 RET 执行。

以下是用于执行内联 PASM 代码的内部 Spin2 过程：

- 保存当前流地址，以便PASM 代码执行后恢复。
- 将方法的前16 个长整型变量（包括所有参数、返回值和局部变量）从集线器RAM 复制到cog 寄存器\$1E0..\$1EF。
- 将内嵌PASM 代码长整型从集线器RAM 复制到cog 寄存器中，从ORG 地址开始（默认为\$000）。
- 调用PASM 代码。
- 将cog 寄存器\$1E0..\$1EF 中的16 个长整型恢复到集线器RAM，以更新任何修改的方法变量。
- 恢复流地址并恢复Spin2 字节码执行。

在您的内联 PASM 代码中，您可以执行以下所有操作：

- 读写以下寄存器区域：
 - \$000..\$12F，您的 PASM 代码加载到其中。您甚至可以在此范围内的不同地址加载不同的 PASM 程序，并从 Spin2 调用它们。
 - \$1D8..\$1DF，这是通用寄存器，名为 PR0..PR7，可用于 PASM 和 Spin2 代码。
 - \$1E0..\$1EF，临时包含该方法的前 16 个长集线器 RAM 变量，并临时分配相同的符号名称。
 - \$1F0..\$1FF，包括 IJMP3、IRET3、IJMP2、IRET2、IJMP1、IRET1、PA、PB、PTRA、PTRB、DIRA、DIRB、OUTA、OUTB、INA 和 INB。
 - 避免写入 \$130..\$1D7 和 LUT RAM，因为 Spin2 解释器占用这些区域。您可以查看“Spin2_interpreter.spin2”来查看解释器代码。
- 暂时使用流光。
- 使用最多 5 层硬件堆栈进行嵌套调用，包括对集线器 RAM 的调用。
- 声明和引用常规符号和局部符号。这些符号在您的 PASM 代码之外无法访问。
- 声明BYTE、WORD 和LONG 数据。
- 使用RES、ORGF 和FIT 指令。内联 PASM 代码中不允许使用指令 ORG、ORGH、ALIGNW、ALIGNL 和 FILE。
- 建立一个中断，执行 cog 寄存器 \$000..\$12F 中剩余的代码。Spin2 适应中断并且仅在必要时短暂地停止它们。
- 通过执行_RET_ 或RET 指令，随时返回Spin2。

从 SPIN2 调用 PASM

您可以在 Spin2 中执行 CALL(address) 以在 cog 寄存器空间或集线器 RAM 中执行 PASM 代码。

```
PUB go() | x

REPEAT
  CALL(@random)
  PINWRITE(56 ADDPINS 7, PR0)
  WAITMS(100)

DAT      ORGH      'hub PASM program to rotate a random bit into pr0

random GETRND WC
_RET_  RCL      PR0,#1
```

以下是执行 CALL 的内部 Spin2 过程：

- 保存当前流地址，以便PASM 代码执行后恢复。
- 调用PASM 代码。
- 恢复流地址并恢复Spin2 字节码执行。

在您调用的代码中，您可以执行以下所有操作：

- 读写以下寄存器区域：
 - \$000..\$12F，其中可能包含您之前加载的 PASM 代码和/或数据。
 - \$1D8..\$1DF，这是通用寄存器，名为 PR0..PR7，可用于 PASM 和 Spin2 代码。
 - \$1E0..\$1EF，可用于暂存器使用，但在 Spin2 恢复时可能会被重写。
 - \$1F0..\$1FF，包括 IJMP3、IRET3、IJMP2、IRET2、IJMP1、IRET1、PA、PB、PTRA、PTRB、DIRA、DIRB、OUTA、OUTB、INA 和 INB。
 - 避免写入 \$130..\$1D7 和 LUT RAM，因为 Spin2 解释器占用这些区域。您可以查看“Spin2_interpreter.spin2”来查看解释器代码。
- 暂时使用流光。
- 使用最多 5 层硬件堆栈进行嵌套调用，包括对集线器 RAM 的调用。
- 建立一个中断，执行 cog 寄存器 \$000..\$12F 中剩余的代码。Spin2 适应中断并且仅在必要时短暂地停止它们。
- 通过执行_RET_ 或RET 指令，随时返回Spin2。

REGLOAD and REGEXEC

Spin2 指令 REGLOAD(HubAddress) 和 REGEXEC(HubAddress) 用于将 PASM 代码和/或数据块从集线器 RAM 加载或加载并执行到 cog 寄存器中。

PASM 代码和/或数据块 前面必须有两个字，它们提供起始寄存器和要加载的寄存器（长整型）数量（负 1）。

```
PUB go()

    REGLOAD(@chunk)      'load self-defined chunk from hub into registers

    REPEAT
        CALL(#start)      'call program within chunk at register address
        WAITMS(100)

DAT

chunk    WORD    start,finish-start-1 'define chunk start and size-1
         ORG     $120                'org can be $000..$130-size

start    DRVNRD   #56 ADDPINS 7        'some code
_ret_    DRVNOT   #0                  'more code + return
finish
```

REGEXEC 的工作方式与 REGLOAD 类似，但它也在加载后调用块的起始寄存器。

在下面的示例中，REGEXEC 在高位寄存器内存中启动一段代码，设置定时器中断，然后返回 Spin2。同时，由于 Spin2 方法每 100ms 重复随机化引脚 60..63，因此加载到高位寄存器内存中的代码块会永久保留定时器中断并每 500ms 切换引脚 56..59。请注意，寄存器 \$000..\$127 仍然可用于其他代码块，并且中断 2 和 3 仍然未使用。

```
PUB go()

    REGEXEC(@chunk)      'load self-defined chunk and execute it
                        'chunk starts timer interrupt and returns

    REPEAT
        PINWRITE(60 ADDPINS 3, GETRND()) 'randomize pins 60..63
        WAITMS(100)          'pins 56..59 toggle via interrupt

DAT

chunk    WORD    start,finish-start-1 'define chunk start and size-1
         ORG     $128                'org can be $000..$130-size

start    MOV     IJMP1,#isr          'set int1 vector
         SETINT1 #1                  'set int1 to ct-passed-ct1 event
         GETCT   PR0                  'get ct
         _ret_   ADDCT1 PR0,bigwait    'set initial ct1 target, return to Spin2

isr       DRVNOT   #56 ADDPINS 3      'interrupt service routine, toggle 56..59
         ADDCT1   PR0,bigwait        'set next ct1 target
         RETI1                      'return from interrupt

bigwait   LONG    20_000_000 / 2     '500ms second on RCFAST
finish
```

调试

Spin2 编译器包含一个隐形调试程序，可以随您的应用程序自动下载。它使用 RAM 的最后 16KB 加上每个 Spin2 DEBUG 语句的几个字节以及每个 PASM DEBUG 语句的一条指令。您可以在应用程序中放置 DEBUG 语句，其中包含输出命令，这些命令将在应用程序运行时串行传输变量和方程的状态。每次执行期间遇到 DEBUG 语句时，都会调用调试器并输出该语句的消息。在 PNut 中通过将 Ctrl 键添加到常用的 F10 来“运行”或将 F11 添加到“程序”来启动调试，或者在 Propeller Tool 中通过使用 Ctrl+D 启用调试模式，然后像平常一样使用 F10 或 F11。这将使用所有 DEBU G 语句编译您的应用程序，将调试器添加到下载中，然后打开“DEBUG 输出”窗口，该窗口在应用程序启动时开始接收消息。

关于 DEBUG 系统的一些知识

● 要使用调试器，必须配置至少一个来自晶振或外部输入的10 MHz 时钟。您不能使用 RCFast 或 RCsLow。● 调试器占用集线器RAM 的顶部16 KB，重新映射到\$FC000..\$FFFF 并写保护。价格为 \$7C000..\$7FFFF 的集线器 RAM 将不再可用。● 定义每个DEBUG 语句的数据存储在RAM 前16 KB 的调试器映像中，从而最大限度地减少对应用程序代码的影响。● 在Spin2 中，每个DEBUG 语句添加三个字节，加上引用变量和解析DEBUG 语句中使用的运行时表达式所需的任何代码。● 在PASM 中，每个DEBUG 语句添加一条指令（长指令）。● 当不编译为DEBUG 模式时，DEBUG 语句将被编译器忽略，因此在不使用调试时无需将它们注释掉。● 如果您的应用程序中不存在DEBUG 语句，您仍会在cogs 启动时收到通知消息。● 通过在常用的 F9..F11 键之前按住 CTRL（在 PNut 中）或使用 CTRL+D（在 Propeller Tool 中）启用调试来调用调试，以进行编译、下载和编程到闪存。● 在执行过程中，当遇到DEBUG 语句时，文本消息将以8-N-1 格式以2 Mbaud 在P62 上串行发送。● DEBUG 消息始终以“CogN”开头，其中N 是齿轮编号，后跟两个空格，并且始终以CR+LF（换行）结束。● 应用程序中最多可以存在255 个DEBUG 语句，因为BRK 指令用于中断和选择特定DEBUG 语句定义。● 您可以定义多个符号来修改调试器行为：DEBUG_COG、DEBUG_DELAY、DEBUG_BAUD、DEBUG_PIN、DEBUG_TIMESTAMP 等。请参见表。● 每次启动启用调试的cog 时，都会输出一条调试消息，以指示cog 编号、代码地址(PTRB)、参数(PTRA)以及“加载”或“跳转”模式。● 对于Spin2，DEBUG 语句可以输出表达式和变量值、集线器字节/字/长数组以及寄存器数组。● 对于PASM，DEBUG 语句可以输出寄存器值/数组、集线器字节/字/长数组和常量。使用 PASM 语法：隐含寄存器或#immediate。● DEBUG 输出数据可以以十进制、十六进制或二进制、有符号或无符号的形式显示，大小可以为字节、字、长整型或自动。还支持集线器字符串。● DEBUG 输出命令显示源和值：“DEBUG(UHEX(x))”可能输出“x = \$123”。● 输出数据的 DEBUG 命令可以有多个参数，以逗号分隔：SDEC(x,y,z) 和 LSTR(ptr1,size1,ptr2,size2)● 数据之间自动输出逗号：“DEBUG(UHEX_BYTE(d,e,f),SDEC(g))”可能会输出“d = \$45, e = \$67, f = \$89, g = -1_024”。● 所有DEBUG 输出命令都有备用版本，以“_”结尾，仅输出值：DEBUG(UHEX_BYTE_(d,e,f)) 可能输出“\$45,\$67,\$89”。● DEBUG 语句除命令外还可以包含以逗号分隔的字符串和字符：DEBUG("We got here! Oh, Nooooo...", 13, 13)● DEBUG 语句可以包含 IF() 和 IFNOT() 命令以控制语句内的进一步输出。初始 IF/IFNOT 将门控整个消息。● DEBUG 语句可能包含最终的 DLY（毫秒）命令来减慢 cog 的消息传递速度，因为消息可能以每秒 ~10,000 条的速率传输。● DEBUG 串行输出可以以不同的波特率重新定向到不同的引脚，以便在其他地方显示/记录。● LOCK[15] 由调试器分配，并在调试中断期间在所有 cog 之间使用，以分时共享 DEBUG 串行发送引脚。● 命令行支持仅调试模式： Pnut -debug {CommPort if not 1} {BaudRate if not 2_000_000}

DEBUG 语句中使用的命令

Conditionals	Details
IF(condition)	If condition <> 0 then continue at the next command within the DEBUG statement, else skip all remaining commands and output CR+LF. If used as the first command in the DEBUG statement, IF will gate ALL output for the statement, including the "CogN "+CR+LF. This way, DEBUG messages can be entirely suppressed, so that you can filter what is important.
IFNOT(condition)	If condition = 0 then continue at the next command within the DEBUG statement, else skip all remaining commands and output CR+LF. If used as the first command in the DEBUG statement, IFNOT will gate ALL output for the statement, including the "CogN "+CR+LF. This way, DEBUG messages can be entirely suppressed, so that you can filter what is important.

String Output *	Details	Output
ZSTR(hub_pointer)	Output zero-terminated string at hub_pointer	"Hello!"
LSTR(hub_pointer,size)	Output 'size' characters of string at hub_pointer	"Goodbye."

Decimal Output, unsigned *	Details	Min Output	Max Output
UDEC(value)	Output unsigned decimal value	0	4_294_967_295
UDEC_BYTE(value)	Output byte-size unsigned decimal value	0	255
UDEC_WORD(value)	Output word-size unsigned decimal value	0	65_535
UDEC_LONG(value)	Output long-size unsigned decimal value	0	4_294_967_295
UDEC_REG_ARRAY(reg_pointer,size)	Output register array as unsigned decimal values	0	4_294_967_295
UDEC_BYTE_ARRAY(hub_pointer,size)	Output hub byte array as unsigned decimal values	0	255
UDEC_WORD_ARRAY(hub_pointer,size)	Output hub word array as unsigned decimal values	0	65_535
UDEC_LONG_ARRAY(hub_pointer,size)	Output hub long array as unsigned decimal values	0	4_294_967_295
Decimal Output, signed *	Details	Min Output	Max Output

SDEC(value) 输出有符号十进制值 -2_147_483_648 2_147_483_647 SDEC_BYTE(value) 输出字节大小有符号十进制值 -32_768 32_767 SDEC_LONG(value) 输出长整型有符号十进制值 -2_147_483_648 2_147_483_647 SDEC_REG_ARRAY(reg_pointer,size) 将寄存器数组输出为有符号十进制值 -2_147_483_648 2_147_483_647 SDEC_BYTE_ARRAY(hub_pointer,size) 将集线器字节数组输出为有符号十进制值 -128 127			
SDEC_WORD_ARRAY(hub_pointer,size)	Output hub word array as signed decimal values	-32_768	32_767
SDEC_LONG_ARRAY(hub_pointer,size)	Output hub long array as signed decimal values	-2_147_483_648	2_147_483_647
Hexadecimal Output, unsigned *	Details	Min Output	Max Output
UHEX(value)	Output auto-size unsigned hex value	\$0	\$FFFF_FFFF
UHEX_BYTE(value)	Output byte-size unsigned hex value	\$00	\$FF
UHEX_WORD(value)	Output word-size unsigned hex value	\$0000	\$FFFF
UHEX_LONG(value)	Output long-size unsigned hex value	\$0000_0000	\$FFFF_FFFF
UHEX_REG_ARRAY(reg_pointer,size)	Output register array as unsigned hex values	\$0000_0000	\$FFFF_FFFF
UHEX_BYTE_ARRAY(hub_pointer,size)	Output hub byte array as unsigned hex values	\$00	\$FF
UHEX_WORD_ARRAY(hub_pointer,size)	Output hub word array as unsigned hex values	\$0000	\$FFFF
UHEX_LONG_ARRAY(hub_pointer,size)	Output hub long array as unsigned hex values	\$0000_0000	\$FFFF_FFFF
Hexadecimal Output, signed *	Details	Min Output	Max Output
SHEX(value)	Output auto-size signed hex value	-\$0000_0000	\$7FFF_FFFF
SHEX_BYTE(value)	Output byte-size signed hex value	-\$80	\$7F
SHEX_WORD(value)	Output word-size signed hex value	-\$8000	\$7FFF
SHEX_LONG(value)	Output long-size signed hex value	-\$8000_0000	\$7FFF_FFFF
SHEX_REG_ARRAY(reg_pointer,size)	Output register array as signed hex values	-\$8000_0000	\$7FFF_FFFF
SHEX_BYTE_ARRAY(hub_pointer,size)	Output hub byte array as signed hex values	-\$80	\$7F
SHEX_WORD_ARRAY(hub_pointer,size)	Output hub word array as signed hex values	-\$8000	\$7FFF
SHEX_LONG_ARRAY(hub_pointer,size)	Output hub long array as signed hex values	-\$8000_0000	\$7FFF_FFFF
Binary Output, unsigned *	Details	Min Output	Max Output
UBIN(value)	Output auto-size unsigned binary value	%0	%11111111_11111111_11111111_11111111
UBIN_BYTE(value)	Output byte-size unsigned binary value	%00000000	%11111111
UBIN_WORD(value)	Output word-size unsigned binary value	%00000000_00000000	%11111111_11111111
UBIN_LONG(value)	Output long-size unsigned binary value	%00000000_00000000_00000000_00000000	%11111111_11111111_11111111_11111111
UBIN_REG_ARRAY(reg_pointer,size)	Output register array as unsigned binary values	%00000000_00000000_00000000_00000000	%11111111_11111111_11111111_11111111
UBIN_BYTE_ARRAY(hub_pointer,size)	Output hub byte array as unsigned binary values	%00000000	%11111111
UBIN_WORD_ARRAY(hub_pointer,size)	Output hub word array as unsigned binary values	%00000000_00000000	%11111111_11111111
UBIN_LONG_ARRAY(hub_pointer,size)	Output hub long array as unsigned binary values	%00000000_00000000_00000000_00000000	%11111111_11111111_11111111_11111111
Binary Output, signed *	Details	Min Output	Max Output
SBIN(value)	Output auto-size signed binary value	-%10000000_00000000_00000000_00000000	%01111111_11111111_11111111_11111111
SBIN_BYTE(value)	Output byte-size signed binary value	-%10000000	%01111111
SBIN_WORD(value)	Output word-size signed binary value	-%10000000_00000000	%01111111_11111111
SBIN_LONG(value)	Output long-size signed binary value	-%10000000_00000000_00000000_00000000	%01111111_11111111_11111111_11111111
SBIN_REG_ARRAY(reg_pointer,size)	Output register array as signed binary values	-%10000000_00000000_00000000_00000000	%01111111_11111111_11111111_11111111
SBIN_BYTE_ARRAY(hub_pointer,size)	Output hub byte array as signed binary values	-%10000000	%01111111
SBIN_WORD_ARRAY(hub_pointer,size)	Output hub word array as signed binary values	-%10000000_00000000	%01111111_11111111
SBIN_LONG_ARRAY(hub_pointer,size)	Output hub long array as signed binary values	-%10000000_00000000_00000000_00000000	%01111111_11111111_11111111_11111111

Delay to Pace Messages	Details
DLY(milliseconds)	Delay for some milliseconds to slow down continuous message outputs for this cog. DLY is only allowed as the last command in a DEBUG statement, since it releases LOCK[15] before the delay, permitting other cogs to capture LOCK[15] so that they may take control of the DEBUG serial-transmit pin and output their own DEBUG messages.

* These commands accept multiple parameters, or multiple sets of parameters. Alternate commands with the same names, but ending in "_", are also available for value-only output (i.e. ZSTR_, , , , LSTR_, UDEC_).

您可以定义来修改 DEBUG 行为的符号

CON Symbol	Default	Purpose
DOWNLOAD_BAUD	2_000_000	Sets the download baud rate.
DEBUG_COGS	%11111111	Selects which cogs have debug interrupts enabled. Bits 7..0 enable debugging interrupts in cogs 7..0.
DEBUG_DELAY	0	Sets a delay in milliseconds before your application runs and begins transmitting DEBUG messages.
DEBUG_PIN	62	Sets the DEBUG serial output pin. For DEBUG windows to open, DEBUG_PIN must be 62.
DEBUG_BAUD	DOWNLOAD_BAUD	Sets the DEBUG baud rate. May be necessary to add DEBUG_DELAY if DEBUG_BAUD is less than DOWNLOAD_BAUD.
DEBUG_TIMESTAMP	undefined	By declaring this symbol, each DEBUG message will be time-stamped with the 64-bit CT value.
DEBUG_LOG_SIZE	0	Sets the maximum size of the 'DEBUG.log' file which will collect DEBUG messages. A value of 0 will inhibit log file generation.
DEBUG_LEFT	(dynamic)	Sets the left screen coordinate where the DEBUG message window will appear.
DEBUG_TOP	(dynamic)	Sets the top screen coordinate where the DEBUG message window will appear.
DEBUG_WIDTH	(dynamic)	Sets the width of the DEBUG message window.
DEBUG_HEIGHT	(dynamic)	Sets the height of the DEBUG message window.
DEBUG_DISPLAY_LEFT	0	Sets the overall left screen offset where any DEBUG displays will appear (adds to 'POS' x coordinate in each DEBUG display).
DEBUG_DISPLAY_TOP	0	Sets the overall top screen offset where any DEBUG displays will appear (adds to 'POS' y coordinate in each DEBUG display).
DEBUG_WINDOWS_OFF	0	Disables any DEBUG windows from opening after downloading, if set to a non-zero value.

Spin2 中的简单调试示例

```
CON _clkfreq = 10_000_000      'set 10 MHz clock (assumes 20 MHz crystal)

PUB go() | i
  REPEAT i FROM 0 TO 9          'count from 0 to 9
    DEBUG(UDEC(i))              'debug, output i
```

当使用 Ctrl-F10 运行时，“调试”窗口将打开，显示如下：

```
Cog0  INIT $0000_0000 $0000_0000 load
Cog0  INIT $0000_0D6C $0000_10BC jump
Cog0  i = 0
Cog0  i = 1
Cog0  i = 2
Cog0  i = 3
Cog0  i = 4
Cog0  i = 5
Cog0  i = 6
Cog0  i = 7
Cog0  i = 8
Cog0  i = 9
```

在报告的第一行中，您可以看到 Cog0 从 \$00000 加载 Spin2 设置代码。在第二行中，Spin2 解释器从 \$00D58 开始启动，其堆栈空间从 \$0101C 开始。之后，Spin2 程序开始运行，您会看到“i”从 0 迭代到 9。

如果将 REPEAT 中的“9”更改为“99”，数据会滚动得太快而无法读取，但通过在 DEBUG 语句末尾添加 DLY 命令，可以减慢输出速度：`debug(udec(i), dly(250)) 'debug, output i with a 250ms delay after every reports`

假设您想要限制输出的消息，以便仅显示“i”的奇数值。您可以在 DEBUG 语句开头使用 IF 来检查“i”的最低有效位。当 IF 为 false 时，不会输出任何消息，导致仅显示 i 的奇数值：

```
debug(if(i & 1), udec(i), dly(250)) '调试，每次报告后仅输出奇数 i 值，延迟 250ms
```

PASM 中的简单 DEBUG 示例

```
CON _clkfreq = 10_000_000      'set 10 MHz clock (assumes 20 MHz crystal)
```

```
DAT      ORG

        MOV      i,#9          'set i to 9
loop     DEBUG    (UHEX_LONG(i)) 'debug, output i in hex
        DJNF     i,#loop        'decrement i and loop if not -1
        JMP      #$             'don't go wandering off, stay here

i        RES      1             'reserve one register as 'i'
```

当使用 Ctrl-F10 运行时，“调试”窗口将打开，显示如下：

```
Cog0  INIT $0000_0000 $0000_0000 load
Cog0  i = $0000_0009
Cog0  i = $0000_0008
Cog0  i = $0000_0007
Cog0  i = $0000_0006
Cog0  i = $0000_0005
Cog0  i = $0000_0004
Cog0  i = $0000_0003
Cog0  i = $0000_0002
Cog0  i = $0000_0001
Cog0  i = $0000_0000
```

在报告的第一行中，您可以看到 Cog0 从 \$00000 加载我们的 PASM 程序。之后，程序运行，您会看到 “i” 从 9 迭代到 0。

如果将 MOV 指令中的 “9” 更改为 “99” 并且希望减慢速度，请在 DEBUG 语句中添加 DLY 命令，并确保将毫秒表示为 #250，因为普通的 250 将被理解为寄存器 250：

```
        MOV      i,#99          'set i to 99
        DEBUG    (UHEX_LONG(i)) '调试，以十六进制输出 i 并在每次报告后延迟 250ms
        DLY      #250           'delay 250ms
```

调试内存利用率

以下是 Spin2 DEBUG 语句的内存利用率。您可以看到，在 Spin2 端，需要一个字节码来读取变量 “i”，然后三个强制字节码组成了实际的 DEBUG 指令。

“堆栈调整” 字节告诉解释器将堆栈放下多远才能有效地 “弹出” 为准备 DEBUG 事件而推送的所有表达式。仅在 “i” 的情况下，堆栈需要减少四个字节（一个长）。当调用调试器时，它所需的值将排序在当前 Spin2 堆栈指针的正上方。

debug("What? ", udec(i))

<i>Spin2 bytecodes in application</i>	<i>DEBUG database in top 16KB of RAM</i>
<pre>\$E0 - read 'i' \$44 - DEBUG bytecode \$04 - stack adjustment \$01 - unique BRK code</pre>	<pre>\$04 - output "CogN " \$06 - output string \$57 - "W" \$68 - "h" \$61 - "a" \$74 - "t" \$3F - "?" \$20 - " " \$00 - end of string \$41 - UDEC + output string \$69 - "i" \$00 - end of string \$00 - end of DEBUG statement</pre>

“唯一的BRK代码” 字节用作索引来查找内存顶部的DEBUG数据库中的特定记录，调试器从中读取其命令。

在这种情况下，DEBUG 处于活动状态，但 cog 已通过 DEBUG_ENABLE 符号禁用其调试中断，Spin2 DEBUG 指令不会触发调试中断，但它们仍然会触发调试中断。
弹出任何DEBUG-in倾向于堆栈中的值，因此这些是无害的事件。

对于 PASM DEBUG 语句，在放置 DEBUG 命令的位置插入 “BRK #code” 指令，并且所有相关数据都驻留在 DEBUG 数据库中。如果 cog 的调试中断被禁用，“BRK #code” 指令将不执行任何操作，需要两个时钟。

调试和中断

在 DEBUG 语句期间收到的中断请求将在 DEBUG 完成后执行，但响应时间可能会出现偏差，导致中断的重新触发设置无法正确发生。高频周期性智能引脚中断特别容易出现此问题。想象一下，您在正常的 ISR（中断服务例程）中执行 AKPIN 指令来降低 INA/INB 信号，以便智能引脚可以使其再次变高，从而触发新的中断。同时，在 AKPIN 之后和 RETIx 之前，智能引脚触发，将 INA/INB 拉高。发生这种情况只是因为您的周期帧时序已偏离 DEBUG 语句。该中断不会被看到，因为它发生在 ISR 繁忙时。这将导致中断停止循环。不过，CT 中断不易出现此问题，因为它们有 \$800 0_0000 个时钟周期可供识别。为了解决智能引脚重新触发问题，您可以在 INA/INB 高电平（而不是 INA/INB 上升电平）上触发，但这可能会导致智能引脚配置出现性能问题。

一种故障安全方法 解决这个 DEBUG/中断困境的方法是仅从不在后台执行 ISR 的 cog 中执行 DEBUG 语句。ISR 是否可以容忍时序偏差并且没有风险 挂起中断循环，您可以执行 DEBUG 语句，并进行一些可理解的中断时序降级。

图形调试显示

DEBUG 消息可以调用内置于工具中的特殊图形 DEBUG 显示。这些图形显示均采用独特窗口的形式。一旦实例化，显示器就可以连续输入数据以生成动画可视化。这些显示对于开发和调试非常方便，因为可以在其本机上下文中查看各种数据类型。最多可同时运行 32 个图形显示器。

当调试时 message 包含一个反引号 (`) 字符 (ASCII \$60)，一个包含从反引号到消息结尾的所有内容的字符串，被发送到图形调试显示解析器。解析器 查找几种不同的元素类型，将任何逗号视为空格：

Element Type	Example	Description
display_type	LOGIC, SCOPE, PLOT, BITMAP	This is the formal name of the graphical DEBUG display type you wish to instantiate.
unknown_symbol	MyLogicDisplay	Each graphical DEBUG display Instance must be given a unique symbolic name.
instance_name	MyLogicDisplay	Once instantiated, a graphical DEBUG display instance is referenced by its symbolic name.
keyword	TITLE, POS, SIZE, SAMPLES	Keywords are used to configure displays. They might be followed by numbers, strings, and other keywords.
number	1024, \$FF, %1010	Numbers can be expressed in decimal, hex (\$), and binary (%).
string	'Here is a string'	Strings are expressed within single-quotes.

在了解所有这些如何组合在一起之前，我们需要先了解一些可用于显示的特殊调试显示语法。当 DEBUG 语句中的第一个字符是反引号时，将调用此语法。这会导致 DEBUG 语句中的所有内容都被视为字符串，除非后续反引号充当“转义”字符以允许正常或简写 DEBUG 命令。

DEBUG Statement (v=100, BYTE[a]=1,2,3,4,5)	DEBUG Message Output	Note
DEBUG(`" LOGIC MyDisplay SAMPLES ", SDEC(v))	Cog0 `LOGIC MyDisplay SAMPLES 100	Regular DEBUG syntax can drive DEBUG displays, but it's not optimal.
DEBUG(` LOGIC MyDisplay SAMPLES 100)	`LOGIC MyDisplay SAMPLES 100	DEBUG-display syntax is simpler and 'CogN' is omitted in the output.
DEBUG(` LOGIC MyDisplay SAMPLES `(v))	`LOGIC MyDisplay SAMPLES 100	Decimal numbers are output using `(value) notation. Short for SDEC_.
DEBUG(` LOGIC MyDisplay SAMPLES `\${v})	`LOGIC MyDisplay SAMPLES \$64	Hex numbers are output using `\${value) notation. Short for UHEX_.
DEBUG(` LOGIC MyDisplay SAMPLES `%v))	`LOGIC MyDisplay SAMPLES %1100100	Binary numbers are output using `%value) notation. Short for UBIN_.
DEBUG(` LOGIC MyDisplay TITLE `'#(v)')	`LOGIC MyDisplay TITLE 'd'	Characters are output using `#(value) notation.
DEBUG(`MyDisplay `UDEEC_BYTE_ARRAY_(@a,5))	`MyDisplay 1, 2, 3, 4, 5	Regular DEBUG commands can follow the backtick, as well.

使用图形调试显示有两个步骤。首先，它们必须被实例化，其次，它们必须被提供：

To Use a Display:	1st	2nd	3rd	4th	Note
First, instantiate it.	`	display_type	unknown_symbol	keyword(s), number(s), string(s)	Unknown_symbol becomes instance_name.
Then, feed it.	`	instance_name(s)	keyword(s), number(s), string(s)		Multiple displays can be fed the same data.

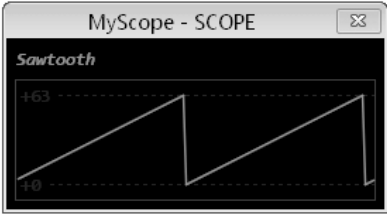
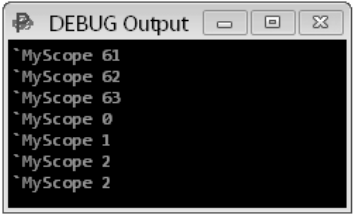
To bring this all together, let's show a sawtooth wave on a SCOPE display:

```
CON _clkfreq = 10_000_000

PUB go() | i

  debug(`SCOPE MyScope SIZE 254 84 SAMPLES 128)
  debug(`MyScope 'Sawtooth' 0 63 64 10 %1111)

  repeat
    debug(`MyScope `(i & 63))
    i++
    waitms(50)
```



在上面的示例中，SCOPE 实例化为 MyScope，大小为 254 x 84 像素，显示 128 个样本。选择 254 的宽度是因为样本编号为 0..127，并且我希望它们以恒定的两像素间距 (127 * 2 = 254) 间隔。选择 84 的高度，以便波形上方和下方各有 10 个像素，波形高度为 64 像素。定义了一个称为“锯齿波”的通道，出于显示目的，其底部值为 0，顶部值为 63，在该范围内高度为 64 像素，并且比示波器窗口底部高出 10 像素。 %1111 启用顶部和底部图例值以及顶线和底线。在 REPEAT 块内，SCOPE 被馈送重复模式 0..63，形成锯齿波。SCOPE 每次收到值时都会更新其显示。如果定义了八个通道（而不是仅一个），它将在每收到八个值时更新显示，绘制所有八个通道。

目前，已实现以下图形调试显示，但将来会添加更多：

Display Types	Descriptions
LOGIC	Logic analyzer with single and multi-bit labels, 1..32 channels, can trigger on pattern
SCOPE	Oscilloscope with 1..8 channels, can trigger on level with hysteresis
SCOPE_XY	XY oscilloscope with 1..8 channels, persistence of 0..512 samples, polar mode, log scale mode
FFT	Fast Fourier Transform with 1..8 channels, 4..2048 points, windowed results, log scale mode
SPECTRO	Spectrograph with 4..2048-point FFT, windowed results, phase-coloring, and log scale mode
PLOT	General-purpose plotter with cartesian and polar modes
TERM	Text terminal with up to 300 x 200 characters, 6..200 point font size, 4 simultaneous color schemes
BITMAP	Bitmap, 1..2048 x 1..2048 pixels, 1/2/4/8/16/32-bit pixels with 19 color systems, 15 direction/autoscroll modes, independent X and Y pixel size of 1..256
MIDI	Piano keyboard with 1..128 keys, velocity depiction, variable screen scale

以下是每种 DEBUG 显示类型的详细说明。

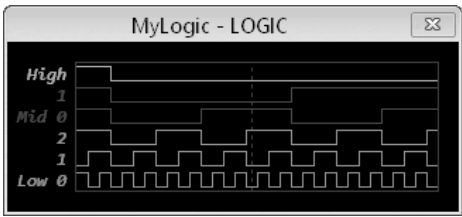
逻辑显示 带有单位和多位标签的逻辑分析仪，1..32 个通道，可以触发模式

```
CON _clkfreq = 10_000_000

PUB go() | i

  debug(`LOGIC MyLogic SAMPLES 32 'Low' 3 'Mid' 2 'High')
  debug(`MyLogic TRIGGER $07 $04 HOLDOFF 2)

  repeat
    debug(`MyLogic `(i & 63))
    i++
    waitms(25)
```



LOGIC Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SAMPLES 4_to_2048	Set the number of samples to track and display.	32
SPACING 2_to_32	Set the sample spacing. The width of the display will be SAMPLES * SPACING.	8
RATE 1_to_2048	Set the number of samples (or triggers, if enabled) before each display update.	1
LINESIZE 1_to_7	Set the line size.	1
TEXTSIZE 6_to_200	Set the legend text size. Height of text determines height of logic levels.	editor text size
COLOR back_color {grid_color}	Set the background and grid colors *.	BLACK, GREY 4
'name' {1_to_32 {color}}	Set the first/next channel or group name, optional bit count, optional color *.	1, default color
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
LOGIC Feeding	Description	Default
TRIGGER mask match sample_offset	Trigger on (data & mask) = match. If mask = 0, trigger is disabled.	0, 1, SAMPLES / 2
HOLDOFF 2_to_2048	Set the minimum number of samples required from trigger to trigger.	SAMPLES
data	Numerical data is applied LSB-first to the channels.	
CLEAR	Clear the sample buffer and display, wait for new data.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* 颜色为 rgb24 值，否则为 BLACK / WHITE 或 0 范围/蓝色/绿色/青色/红色/品红色/黄色/灰色，后跟亮度可选 0..15（默认值为 8）。

逻辑显示器可用于显示高速捕获的数据。在下面的示例中，P2 使用智能引脚以 333 Mbaud 生成 8-N-1 串行。该比特流可以由流传输器捕获。在每个时钟上，流传输器都会记录从相邻引脚读取的智能引脚的 IN 信号及其输出状态。每次它获得四个两位样本集时，它都会执行一个 RFBYTE 将它们保存到集线器 RAM 中，形成连续的字节、字和长整型。通过调用 LONGS_2BIT 打包数据模式，我们可以让 LOGIC 显示从 long 中解包两位样本集，每个 long 产生 16 个集合。

```
CON _clkfreq = 333_333_333 'go really fast, 3ns clock period
  rxpin = 24 'even pin
  txpin = rxpin+1 'odd pin
  samps = 32 'multiple of 16 samples
  bufflongs = samps / 16 'each long holds 16 2-bit samples
  xmode = $D0800000 + rxpin << 17 + samps 'streamer mode

VAR buff[bufflongs]

PUB go() | i, buffaddr

  debug(`logic Serial samples `(samps) spacing 12 'TX' 'IN' longs_2bit)
  debug(`Serial trigger %10 %10 22)
  buffaddr := @buff

  repeat
    org
    wrpin ##+1<<28,#rxpin 'rxpin inputs txpin at rxpin+1

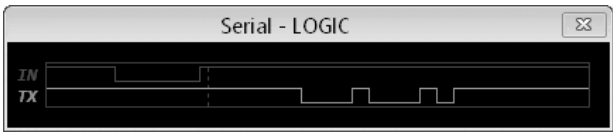
    wrpin ##01_11110_0,#txpin 'set async tx mode for txpin
    wxpin ##1<<16+8-1,#txpin 'set baud=sysclock/1 and size=8
    dirh #txpin 'enable smart pin

    wrfast #0,buffaddr 'set write-fast at buff
    xinit ##xmode,#0 'start capturing 2-bit samples

    wy pin i,#txpin 'transmit serial byte

    waitxfi 'wait for streamer capture done
  end

  debug(`Serial `uhex_long_array_(@buff, bufflongs))
  i++
  waitms(20)
```



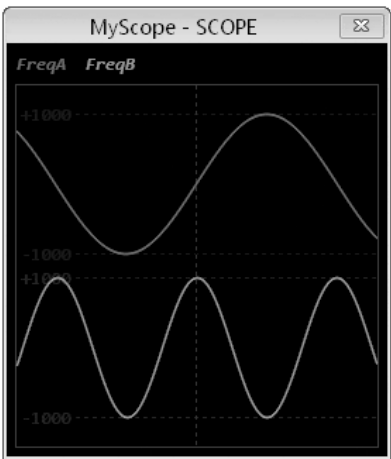
SCOPE Display Oscilloscope with 1..8 channels, can trigger on level with hysteresis

```
CON _clkfreq = 100_000_000

PUB go() | a, af, b, bf

debug(`SCOPE MyScope)
debug(`MyScope 'FreqA' -1000 1000 100 136 15 MAGENTA)
debug(`MyScope 'FreqB' -1000 1000 100 20 15 ORANGE)
debug(`MyScope TRIGGER 0 HOLDOFF 2)

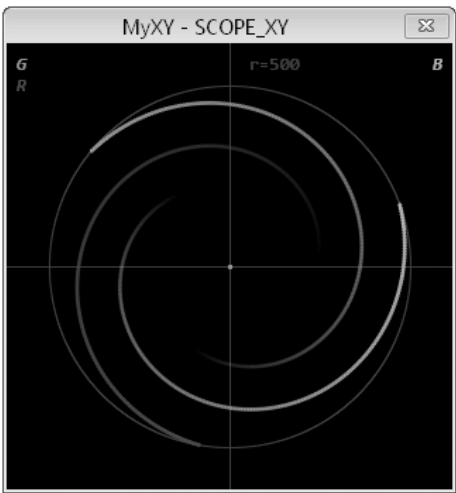
repeat
  a := qsin(1000, af++, 200)
  b := qsin(1000, bf++, 99)
  debug(`MyScope `(a,b))
  waitus(200)
```



SCOPE Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE width height	Set the display size (32..2048 x 32..2048)	255, 256
SAMPLES 16_to_2048	Set the number of samples to track and display.	256
RATE 1_to_2048	Set the number of samples (or triggers, if enabled) before each display update.	1
DOTSIZE 0_to_32	Set the dot size in pixels for showing exact sample points.	0
LINESIZE 0_to_32	Set the line size in half-pixels for connecting sample points.	3
TEXTSIZE 6_to_200	Set the legend text size.	editor text size
COLOR back_color {grid_color}	Set the background and grid colors *.	BLACK, GREY 4
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
SCOPE Feeding	Description	Default
'name' {min {max {y_size {y_base {legend {color}}}}}}	Set first/next channel name, min value, max value, y size, y base, legend, and color *. Legend is %abcd, where %a to %d enable max legend, min legend, max line, min line.	full, no legend, default color
TRIGGER channel {arm_level {trigger_level {offset}}}	Set the trigger channel, arm level, trigger level, and right offset. If channel=-1, disabled.	-1, -1, 0, width / 2
HOLDOFF 2_to_2048	Set the minimum number of samples required from trigger to trigger.	SAMPLES
data	Numerical data is applied to the channels in ascending order.	
CLEAR	Clear the sample buffer and display, wait for new data.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* 颜色为rgb24值, 否则为黑/白或橙 E / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GRAY 后跟一个选项 亮度最终为 0..15 (默认值为 8) 。

```
CON _clkfreq = 100_000_000 PUB go() | i debug(SCOPE_XY MyXY R
ANGE 500 POLAR 360 'G' 'R' 'B') 重复重复 i 从 0 到 500 debug(MyXY
` (i, i, i, i+120, i, i+240)) 等待时间(5)
```

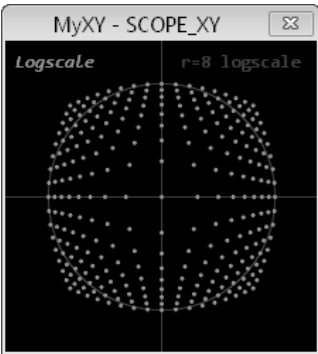
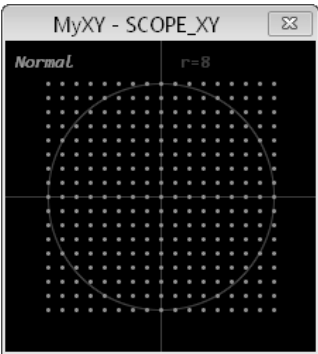


SCOPE_XY Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE radius	Set the display radius in pixels.	128
RANGE 1_to_7FFFFFFF	Set the unit circle radius for incoming data	\$7FFFFFFF
SAMPLES 0_to_512	Set the number of samples to track and display with persistence. Use 0 for infinite persistence.	256
RATE 1_to_512	Set the number of samples before each display update.	1
DOTSIZE 2_to_20	Set the dot size in half-pixels for showing sample points.	6
TEXTSIZE 6_to_200	Set the legend text size.	editor text size
COLOR back_color {grid_color}	Set the background and grid colors *.	BLACK, GREY 4
POLAR {twopi {offset}}	Set polar mode, twopi value, and offset. For a twopi value of \$100000000 or -\$100000000, use 0 or -1.	\$100000000, 0
LOGSCALE	Set log-scale mode to magnify points within the unit circle.	<off>
'name' {color}	Set the first/next channel name and optionally assign it a color *.	default color
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
SCOPE_XY Feeding	Description	Default
x y	X-Y data pairs are applied to the channels in ascending order. In polar mode, x=length and y=angle.	
CLEAR	Clear the sample buffer and display, wait for new data.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

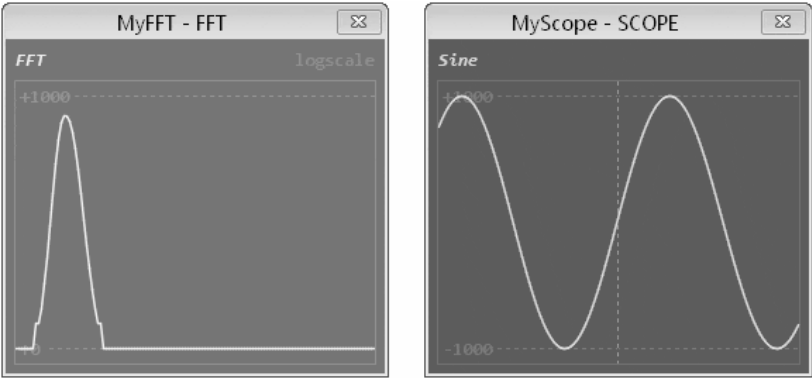
*颜色为 rgb24 值，其他为黑色/白色或橙色/蓝色/绿色/青色/红色/洋红色/黄色/灰色，后跟可选的 0..15 亮度（默认值 i 节）。CON _clkfreq = 10_000_000 '正常模式' PUB go() | x, y debug(SCOPE_XY MyXY SIZE 80 RANGE 8 SAMPLES 0 '正常') 从 -8 到 8 重复 x 从 -8 到 8 重复 y debug(MyXY `(x,y))

CON _clkfreq = 10_000_000 'LOGSCALE 模式放大低级细节' PUB go() | x, y debug(SCOPE_XY MyXY SIZE 80 RANGE 8 SAMPLES 0 LOGSCALE 'Logscale') 从 -8 到 8 重复 x 从 -8 到 8 重复 y 8 调试(MyXY `(x,y))

第 8



```
CON_clkfreq = 100_000_000 PUB go() | i,j,k ' 设置 FFT 调试(FFT MyFFT SIZE 250 200 SAMPLES 2048 0 127 RATE 2
56 LOGSCALE COLOR YELLOW 4 YELLOW 5) debug(MyFFT 'FFT' 0 1000 180 10 15 YELLOW 12)' 设置 SCOPE debu
g(scope MyScope POS 300 0 SIZE 255 200 COLOR CYAN 4 CYAN 5) debug(MyScope 'Sine' -1000 1000 180 10 15 CYAN
12)调试(MyScope TRIGGER 0) 重复 j += 1550 + qsin(1300, i++, 31_000) k := qsin(1000, j, 50_000) 调试(MyFFT MyScope
` (k)) waitus(100)
```



FFT Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE width height	Set the display size (32..2048 x 32..2048)	256, 256
SAMPLES 4_to_2048 {first {last}}	Set the 2 ⁿ number of FFT inputs points, plus the first and last result values to display.	512, 0, 255
RATE 1_to_2048	Set the number of samples before each display update.	SAMPLES
DOTSIZE 0_to_32	Set the dot size in pixels for showing exact sample points.	0
LINESIZE neg32_to_32	Set the line size in half-pixels for connecting sample points. A negative line size will make isolated vertical lines.	3
TEXTSIZE 6_to_200	Set the legend text size.	editor text size
COLOR back_color {grid_color}	Set the background and grid colors *.	BLACK, GREY 4
LOGSCALE	Set log-scale mode to magnify low-level results.	<off>
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
FFT Feeding	Description	Default
'name' {mag {max {y_size {y_base {legend {color}}}}}}	Set the first/next channel name, magnification factor (2 ⁿ , n = 0..11), max amplitude, y size, y base, legend, and color *. Legend is %abcd, where %a to %d enable max legend, min legend, max line, min line.	full, no legend, default color
data	Numerical data is fed into the channels' sliding Hanning windows from which the FFT computes power levels.	
CLEAR	Clear the sample buffer and display, wait for new data.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* 颜色为 rgb24 值，否则为黑色/白色或橙色/蓝色/绿色/青色/红色/品红色/黄色/灰色，后跟可选的 0..15 亮度（默认值为 8）。

SPECTRO Display Spectrograph with 4..2048-point FFT, phase-coloring, and log scale mode

```
CON _clkfreq = 100_000_000

PUB go() | i, j, k

' Set up SPECTRO
debug(`SPECTRO MySpectro SAMPLES 2048 0 236 RANGE 1000 LUMA8X GREEN)

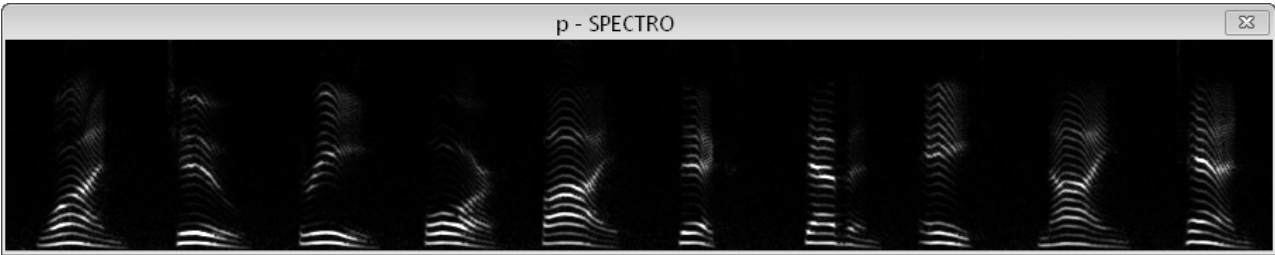
' Set up SCOPE
debug(`SCOPE MyScope POS 280 SIZE 150 200 COLOR GREEN 15 GREEN 12)
debug(`MyScope 'Sine' -1000 1000 180 10 0 GREEN 6)
debug(`MyScope TRIGGER 0)

repeat
  j += 2850 + qsin(2500, i++, 30_000)
  k := qsin(1000, j, 50_000)
  debug(' MySpectro MyScope `(k))
  waitus(100)
```

SPECTRO Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SAMPLES 4_to_2048 {first {last}}	Set the 2 ⁿ number of FFT input points, plus the first and last result values to display (defines display height).	512, 0, 255
DEPTH 1_to_2048	Set the number of vertical-line FFT results to display (defines the display width).	256
MAG 0_to_11	Set the magnification factor (2 ⁿ , n = 0..11).	0
RANGE saturation_power	Set the power level at which pixel brightness saturates.	\$7FFFFFFF
RATE 1_to_2048	Set the number of samples before each display update.	SAMPLES / 8
TRACE 0_to_15	Set the trace pattern (see TRACE animation in BITMAP Display).	15 (right, up, scroll)
DOTSIZE width_and_height {height}	Set the spectrograph pixel-width and pixel-height together, or set them independently.	1, 1
luma_or_hsv {color_or_phase}	Set the color scheme to LUMA8(W)(X) with color *, or HSV16(W)(X) with 0..255 phase-coloring offset.	LUMA8X ORANGE
LOGSCALE	Set log-scale mode to magnify low-level results.	<off>
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
SPECTRO Feeding	Description	Default
data	Numerical data is fed into a sliding Hanning window from which the FFT computes power and phase.	
CLEAR	Clear the sample buffer and display, wait for new data.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* 颜色为橙色/蓝色/绿色/青色/红色/洋红色/黄色/灰色。

下面，SPECTRO 显示器通过连接到麦克风的引脚输入 ADC 样本。这就是从“1”到“10”口头数数的光谱。“1”在左边，“10”在右边。水平趋势线之间的垂直距离是声门音高。较大的亮度趋势是声音共振峰。这给出了我们的耳朵如何感知语音的一些想法：



```
CON _clkfreq = 10_000_000

PUB go(): i, j, k

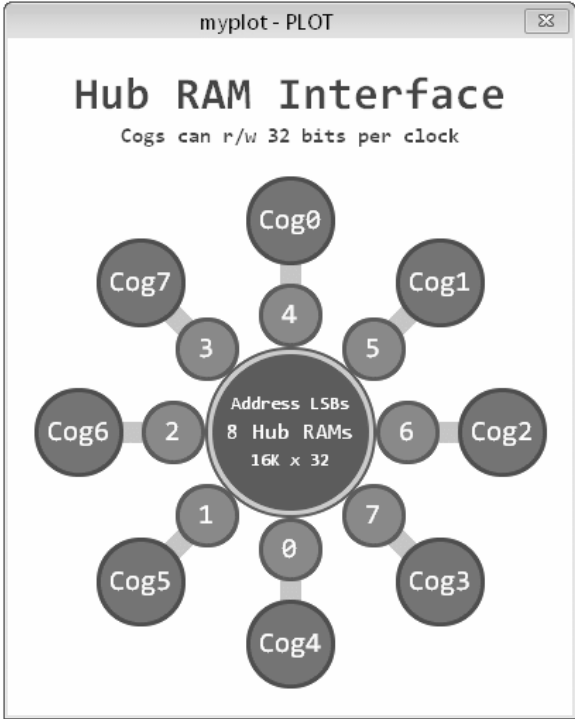
debug(`myplot myplot size 400 480 bgcolor white update)
debug(`myplot origin 200 200 polar -64 -16)
k~
repeat
  debug(`myplot clear)
  debug(`myplot set 240 0 cyan 3 text 24 3 'Hub RAM Interface')
  debug(`myplot set 210 0 text 11 3 'Cogs can r/w 32 bits per clock')

  if k & 8    'move RAMs or draw spokes?
    j++
  else
    repeat i from 0 to 7
      debug(`myplot grey 12 set 83 `(i*8) line 150 `(i*8) 15)

    debug(`myplot set 0 0 cyan 4 circle 121 yellow 7 circle 117 3)
    debug(`myplot set 20 0 white text 10 'Address LSBs')
    debug(`myplot set 0 0 text 12 1 '8 Hub RAMs')
    debug(`myplot set 20 32 text 10 '16K x 32' )

    repeat i from 0 to 7    'draw RAMs and cogs
      debug(`myplot cyan 6 set 83 `(i*8-j) circle 43 text 14 `(i)')
      debug(`myplot cyan 4 set 83 `(i*8-j) circle 45 3)
      debug(`myplot orange 6 set 150 `(i*8) circle 61 text 13 'Cog`(i)')
      debug(`myplot orange 4 set 150 `(i*8) circle 63 3)

    debug(`myplot update `dly(30))
  k++
```



PLOT Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE width height	Set the display width (32..2048) and height (32..2048).	256, 256
DOTSIZE width_and_height {height}	Set the display pixel-width and pixel-height together, or set them independently.	1, 1
lut1_to_rgb24	Set the color mode.	RGB24
LUTCOLORS rgb24 rgb24 ...	For LUT1..LUT8 color modes, load the LUT with rgb24 colors. Use HEX_LONG_ARRAY_ to load colors.	default colors 0..7
BACKCOLOR color	Set the background color according to the current color mode. *	BLACK
UPDATE	Set UPDATE mode. The display will only be updated when fed an 'UPDATE' command.	automatic update
PLOT Feeding	Description	Default
lut1_to_rgb24	Set color mode.	rgb24
LUTCOLORS rgb24 rgb24 ...	For LUT1..LUT8 color modes, load the LUT with rgb24 colors. Use HEX_LONG_ARRAY_ to load values.	default colors 0..7
BACKCOLOR color	Set the background color according to the current color mode. *	BLACK
COLOR color	Set the drawing color according to the current color mode. Use just before TEXT to change text color. *	CYAN
BLACK/WHITE or ORANGE/BLUE/GREEN/CYAN/RED/MAGENTA/YELLOW/GREY {brightness}	Set the drawing color and optional 0..15 brightness for ORANGE..GREY colors (default is 8).	CYAN
OPACITY level	Set the opacity level for DOT, LINE, CIRCLE, OVAL, BOX, and OBOX drawing. 0..255 = clear..opaque.	255
PRECISE	Toggle precise mode, where line size and (x,y) for DOT and LINE are expressed in 256ths of a pixel.	disabled
LINESIZE size	Set the line size in pixels for DOT and LINE drawing.	1
ORIGIN {x_pos y_pos}	Set the origin point to cartesian (x_pos, y_pos) or to the current (x, y) if no values are specified.	0, 0
SET x y	Set the drawing position to (x, y). After LINE, the endpoint becomes the new drawing position.	
DOT {linesize {opacity}}	Draw a dot at the current position with optional LINESIZE and OPACITY overrides.	
LINE x y {linesize {opacity}}	Draw a line from the current position to (x,y) with optional LINESIZE and OPACITY overrides.	
CIRCLE diameter {linesize {opacity}}	Draw a circle around the current position with optional line size (none/0 = solid) and OPACITY override.	
OVAL width height {linesize {opacity}}	Draw an oval around the current position with optional line size (none/0 = solid) and OPACITY override.	
BOX width height {linesize {opacity}}	Draw a box around the current position with optional line size (none/0 = solid) and OPACITY override..	
OBOX width height x_radius y_radius {linesize {opacity}}	Draw a rounded box around the current position with width, height, x and y radii, and optional line size (none/0 = solid) and OPACITY override.	
TEXTSIZE size	Set the text size (6..200).	10
TEXTSTYLE style_YYXXUIWW	Set the text style to %YYXXUIWW: %YY is vertical justification: %00 = middle, %10 = bottom, %11 = top.	%00000001

	%XX is horizontal justification: %00 = middle, %10 = right, %11 = left. %U is underline: %1 = underline. %I is italic: %1 = italic. %WW is weight: %00 = light, %01 = normal, %10 = bold, and %11 = heavy.	
TEXTANGLE angle	Set the text angle. In cartesian mode, the angle is in degrees.	0
TEXT {size {style {angle}}} 'text'	Draw text with overrides for size, style, and angle. To change text color, declare a color just before TEXT.	
POLAR {twopi {offset}}	Set polar mode, twopi value, and offset. For example, POLAR -12 -3 would be like a clock face. For a twopi value of \$100000000 or -\$100000000, use 0 or -1. In polar mode, (x, y) coordinates are interpreted as (length, angle).	\$100000000, 0
CARTESIAN	Set cartesian mode. This is the default mode.	
CLEAR	Clear the plot to the background color.	
UPDATE	Update the window with the current plot. Used in UPDATE mode.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

*颜色是模态值，否则为黑色/白色或橙色/蓝色/绿色/青色/红色/洋红色/黄色/灰色，后跟可选的 0..15 亮度（默认值为 8）。

TERM 用于显示文本的显示终端

CON_clkfreq = 10_000_000

```
PUB go() | i
    debug(`TERM MyTerm SIZE 9 1 TEXTSIZE 40)
    repeat
        repeat i from 50 to 60
            debug(`MyTerm 1 'Temp = `(i)')
            waitms(500)
```



TERM Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE columns rows	Set the number of terminal columns (1..256) and terminal rows (1..256).	40, 20
TEXTSIZE size	Set the terminal text size (6..200).	editor text size
COLOR text_color back_color ...	Set text-color and background-color combos #0.#3. *	default colors
BACKCOLOR color	Set the display background color. *	BLACK
UPDATE	Set UPDATE mode. The display will only be updated when fed an 'UPDATE' command.	automatic update
TERM Feeding	Description	Default
character	0 = Clear terminal display and home cursor. 1 = Home cursor. 2 = Set column to next character value. 3 = Set row to next character value. 4 = Select color combo #0. 5 = Select color combo #1. 6 = Select color combo #2. 7 = Select color combo #3. 8 = Backspace. 9 = Tab to next 8th column. 13+10 or 13 or 10 = New line. 32..255 = Printable character.	
'string'	Print string.	
CLEAR	Clear the display to the background color.	
UPDATE	Update the window with the current text screen. Used in UPDATE mode.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

* 颜色是模态值，否则为 BLACK / WHITE 或 ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GRAY，后跟可选的 0..15 亮度（默认值为 8）。

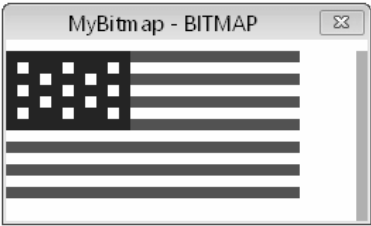
```
CON _clkfreq = 10_000_000

PUB go() | i

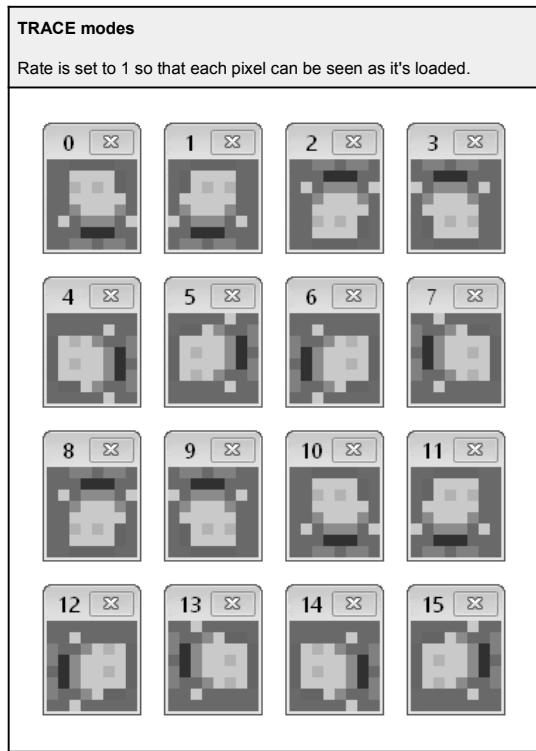
  debug(`bitmap MyBitmap SIZE 32 16 DOTSIZE 8 LUT2 LONGS_2BIT)
  debug(`MyBitmap TRACE 14 LUTCOLORS WHITE RED BLUE YELLOW 6)
  repeat
    debug(`MyBitmap `uhex_(flag[i++ & $1F]) `dly(100))

DAT

flag  long  %%3333333333333330
      long  %%0010101022222220
      long  %%0010101020202020
      long  %%0010101022222220
      long  %%0010101022020220
      long  %%0010101022222220
      long  %%0010101020202020
      long  %%0010101022222220
      long  %%0010101022020220
      long  %%0010101022222220
      long  %%0010101020202020
      long  %%0010101022222220
      long  %%0010101010101010
      long  %%0010101010101010
      long  %%0010101010101010
      long  %%0010101010101010
      long  %%0010101010101010
      long  %%0010101010101010
      long  %%0010101010101010
      long  %%0010101010101010
      long  %%0010101010101010
      long  %%0010101010101010
      long  %%0010101010101010
      long  %%0010101010101010
      long  %%0010101010101010
      long  %%0000000000000000
      long  %%0000000000000000
      long  %%0000000000000000
      long  %%0000000000000000
      long  %%0000000000000000
```

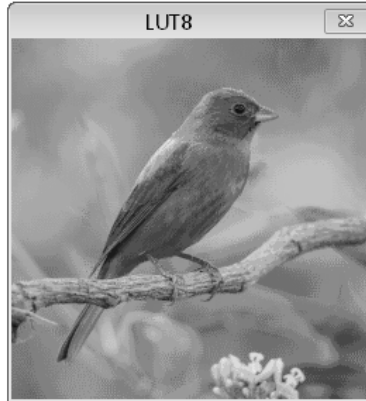


BITMAP Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE x_pixels y_pixels	Set the number of pixels in the bitmap (1..2048 for both x and y).	256, 256
DOTSIZE x_and_y_size {y_size}	Set the size of the displayed pixels (1..256). 'DOTSIZE 16' makes each pixel 16x16 on the final display.	1, 1
lut1_to_rgb24	Set the color mode. See images below.	RGB24
LUTCOLORS rgb24 rgb24 ...	For LUT1..LUT8 color modes, load the LUT with RGB24 colors. Use HEX_LONG_ARRAY_ to load.	default colors 0..7
TRACE 0_to_15	Set the pixel loading direction and whether to scroll after each line is filled. See animation below.	0
RATE pixels_per_update	Set the number of pixels before each display update. 'RATE -1' sets the rate to the bitmap size.	line size
packed_data_mode	Enable packed-data mode. See description at end of this section.	<none>
UPDATE	Set UPDATE mode. The display will only be updated when fed an 'UPDATE' command.	automatic update
BITMAP Feeding	Description	Default
lut1_to_rgb24	Change the color mode.	RGB24
LUTCOLORS rgb24 rgb24 ...	For LUT1..LUT8 color modes, load the LUT with rgb24 colors. Use HEX_LONG_ARRAY_ to load colors.	default colors 0..7
TRACE 0_to_15	Change the direction in which pixels are loaded into the bitmap. Sets the rate to the line size.	0
RATE pixels_per_update	Set the number of pixels before each display update. 'RATE -1' sets the rate to the bitmap size.	
SET x_position {y_position}	Set the current pixel-loading position. Cancels scroll mode by clearing bit 3 of TRACE.	
SCROLL x_scroll y_scroll	Scroll the bitmap by some number of pixels. Negative/positive values determine the direction, 0 = none.	
CLEAR	Clear the bitmap to zero-value pixels.	
UPDATE	Update the window with the current bitmap. Used in UPDATE mode.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the bitmap at 1x scale.	
CLOSE	Close the window.	



Color Mode	Bits/ Pixel	Description	Intention
LUT1	1	Pixel indexes LUT colors 0..1	Memory-efficient 2-color-palette graphics
LUT2	2	Pixel indexes LUT colors 0..3	Memory-efficient 4-color-palette graphics
LUT4	4	Pixel indexes LUT colors 0..15	Memory-efficient 16-color-palette graphics
LUT8	8	Pixel indexes LUT colors 0..255	Memory-efficient 256-color-palette graphics
LUMA8	8	From black to color *	Instrumentation where luminance indicates level
LUMA8W	8	From white to color *	Instrumentation where saturation indicates level
LUMA8X	8	From black to color * to white	Instrumentation where luminance indicates level, peaking in white
HSV8	8	From black to color: %HHHHSSSS	16 hues with 16 luminance levels
HSV8W	8	From white to color: %HHHHSSSS	16 hues with 16 saturation levels, coming from white
HSV8X	8	From black to color to white: %HHHHSSSS	16 hues with 16 luminance levels, peaking in white
RGBI8	8	From black to color: %RGBIIII	8 basic colors with 32 luminance levels
RGBI8W	8	From white to color: %RGBIIII	8 basic colors with 32 saturation levels, coming from white
RGBI8X	8	From black to color to white: %RGBIIII	8 basic colors with 32 luminance levels, peaking in white
RGB8	8	%RRRGGBBB	Byte-level RGB with 8 red, 8 green, and 4 blue levels
HSV16	16	From black to color: %HHHHHHHH_SSSSSSSS	256 hues with 256 luminance levels
HSV16W	16	From white to color: %HHHHHHHH_SSSSSSSS	256 hues with 256 saturation levels, coming from white
HSV16X	16	From black to color to white: %HHHHHHHH_SSSSSSSS	256 hues with 256 luminance levels, peaking in white
RGB16	16	%RRRRRRGG_GGGBBBBB	Word-level RGB with 32 red levels, 64 green levels, and 32 blue levels
RGB24	24	%RRRRRRRR_GGGGGGGG_BBBBBBBB	Full RGB with 256 levels for red, green, and blue

* 颜色为橙色/蓝色/绿色/青色/红色/洋红色/黄色/灰色。



```
CON _clkfreq = 100_000_000

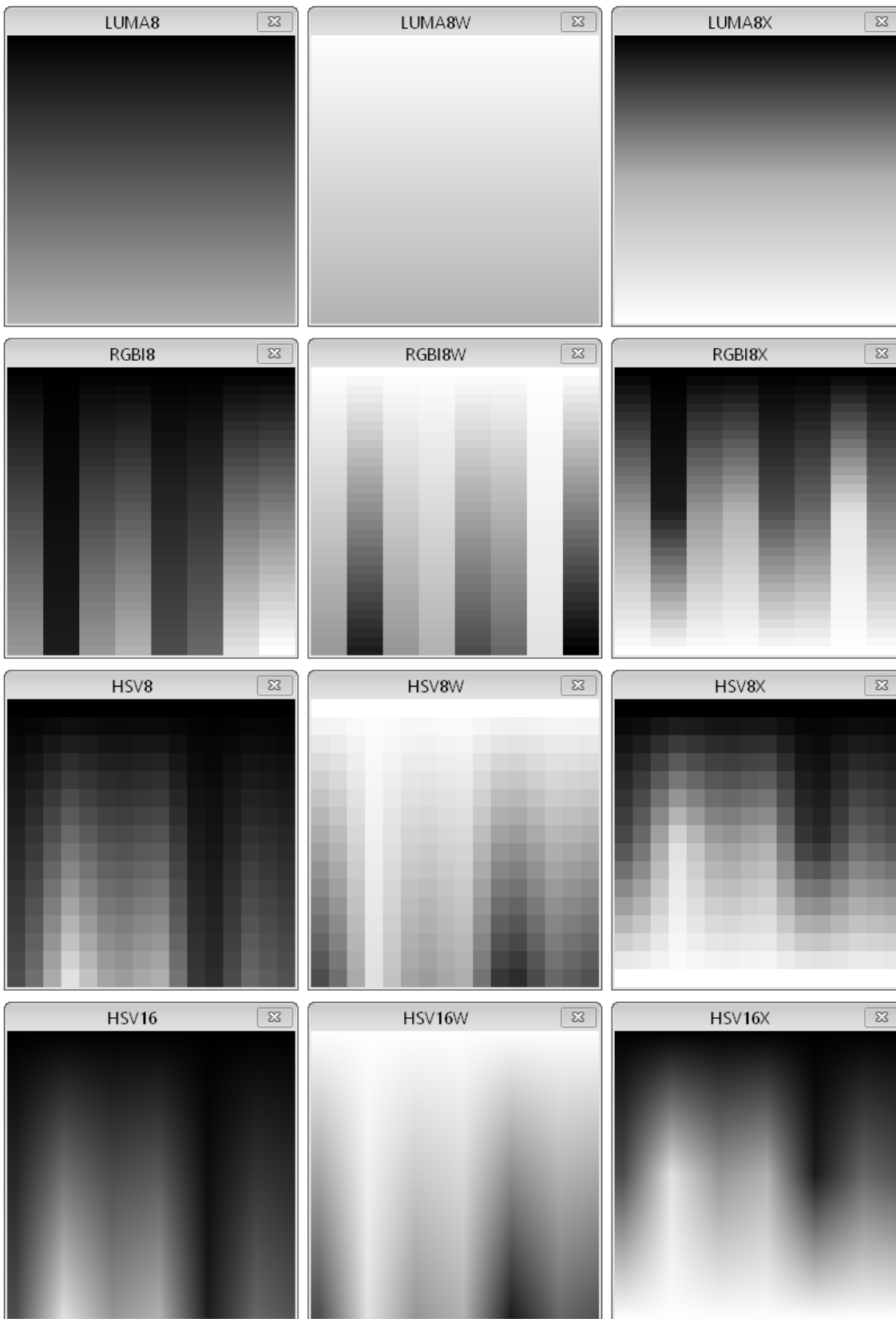
PUB go() | i
  debug('bitmap a title 'LUT1' pos 100 100 trace 2 lut1 longs_1bit alt)
  debug('bitmap b title 'LUT2' pos 370 100 trace 2 lut2 longs_2bit alt)
  debug('bitmap c title 'LUT4' pos 100 395 trace 2 lut4 longs_4bit alt)
  debug('bitmap d title 'LUT8' pos 370 395 trace 2 lut8 longs_8bit)
  debug('bitmap e title 'RGB8' pos 100 690 trace 2 rgb8)
  debug('bitmap f title 'RGB16' pos 370 690 trace 2 rgb16)
  debug('bitmap g title 'RGB24' pos 640 690 trace 2 rgb24)
  waitms(1000)

  showbmp("a", @image1, $8A, 2, $800) 'send LUT1 image
  showbmp("b", @image2, $36, 4, $1000) 'send LUT2 image
  showbmp("c", @image3, $8A, 16, $2000) 'send LUT4 image
  showbmp("d", @image4, $36, 256, $4000) 'send LUT8 image

  i := @image5 + $36 'send RGB8/RGB16/RGB24 images from the same 24-bpp file
  repeat $10000
    debug('e `uhex_(byte[i+0] >> 6 + byte[i+1] >> 5 << 2 + byte[i+2] >> 5 << 5 ))
    debug('f `uhex_(byte[i+0] >> 3 + byte[i+1] >> 2 << 5 + byte[i+2] >> 3 << 11))
    debug('g `uhex_(byte[i+0] + byte[i+1] << 8 + byte[i+2] << 16 ))
    i += 3

PRI showbmp(letter, image_address, lut_offset, lut_size, image_long) | i
  image_address += lut_offset
  debug(`#(letter) lutcolors `uhex_long_array_(image_address, lut_size))
  image_address += lut_size << 2 - 4
  repeat image_long
    debug(`#(letter) `uhex_(long[image_address += 4]))

DAT
image1 file "bird_lut1.bmp"
image2 file "bird_lut2.bmp"
image3 file "bird_lut4.bmp"
image4 file "bird_lut8.bmp"
image5 file "bird_rgb24.bmp"
```



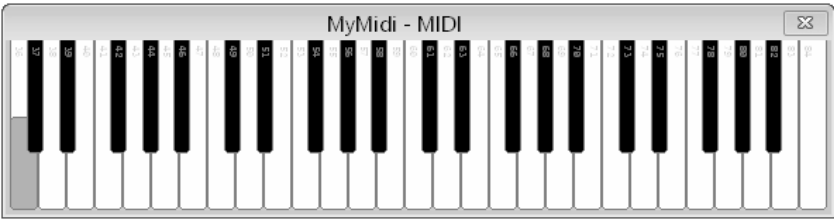
```
CON _clkfreq = 100_000_000
```

```
PUB go() | i
  debug('bitmap a title 'LUMA8' pos 100 100 size 1 256 dotsize 256 1 luma8 cyan)
  debug('bitmap b title 'LUMA8W' pos 370 100 size 1 256 dotsize 256 1 luma8w cyan)
  debug('bitmap c title 'LUMA8X' pos 640 100 size 1 256 dotsize 256 1 luma8x cyan)
  debug('bitmap d title 'RGBI8' pos 100 395 size 8 32 dotsize 32 8 trace 4 rgbi8)
  debug('bitmap e title 'RGBI8W' pos 370 395 size 8 32 dotsize 32 8 trace 4 rgbi8w)
  debug('bitmap f title 'RGBI8X' pos 640 395 size 8 32 dotsize 32 8 trace 4 rgbi8x)
  debug('bitmap g title 'HSV8' pos 100 690 size 16 16 trace 4 dotsize 16 hsv8)
  debug('bitmap h title 'HSV8W' pos 370 690 size 16 16 trace 4 dotsize 16 hsv8w)
  debug('bitmap i title 'HSV8X' pos 640 690 size 16 16 trace 4 dotsize 16 hsv8x)
  debug('bitmap j title 'HSV16' pos 100 985 size 256 256 trace 4 hsv16)
  debug('bitmap k title 'HSV16W' pos 370 985 size 256 256 trace 4 hsv16w)
  debug('bitmap l title 'HSV16X' pos 640 985 size 256 256 trace 4 hsv16x)
  waitms(1000)
  repeat i from 0 to 255
    debug('a b c d e f g h i `uhex_(i))' feed 8-bit displays
  repeat i from 0 to 65535
    debug('j k l `uhex_(i))' feed 16-bit displays
```

```
CON _clkfreq = 10_000_000

PUB go() | i

  debug(`midi MyMidi size 3 range 36 84)
  repeat
    repeat i from 36 to 84
      debug(`MyMidi $90 `(i, getrnd() & $7F))
      waitms(150)
      debug(`MyMidi $80 `(i, 0))
```



MIDI Instantiation	Description	Default
TITLE 'string'	Set the window caption to 'string'.	<none>
POS left top	Set the window position.	0, 0
SIZE keyboard_size	Set the size of the MIDI keyboard display (1..50).	4
RANGE first_key last_key	Set the first and last MIDI key numbers (0..127).	21, 108 (88 keys)
CHANNEL channel_number	Set the MIDI channel number to observe (0..15).	0
COLOR white_key black_key	Set the 'ON' colors for white and black keys. *	CYAN, MAGENTA
MIDI Feeding	Description	Default
byte	If \$90 + channel then NOTE_ON mode, else if \$80 + channel then NOTE_OFF mode. If NOTE_ON mode then receive a key (\$00..\$7F) and then its velocity (\$00..\$7F), update display. If NOTE_OFF mode then receive a key (\$00..\$7F) and then its velocity (\$00..\$7F), update display.	
CLEAR	Clear all notes.	
SAVE {WINDOW} 'filename'	Save a bitmap file (.bmp) of either the entire window or just the display area.	
CLOSE	Close the window.	

*颜色为黑色/白色或橙色/蓝色/绿色/青色/红色/洋红色/黄色/灰色，后跟可选的 0..15 亮度（默认值为 8）。

这是一个 PASM 程序，它接收 P16 上的 MIDI 串行并将其发送到 MIDI 显示器：

```
CON _clkfreq = 10_000_000
rxpin = 16

DAT org

  asmcclk
  debug (`midi m size 2)

  wrpin 11111_0,rxpin
  wxpin ##(clkfreq_/31250) << 16 + 8-1, #rxpin
  drvl #rxpin

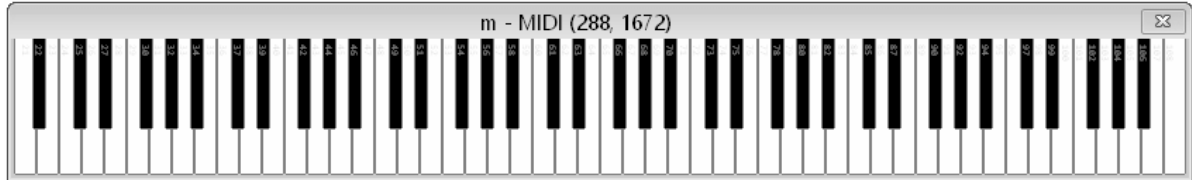
.wait testp #rxpin wc
if_nc jmp #.wait

  rdpin x,#rxpin
  shr x,#32-8

  debug ("`m ", uhex_byte_(x))

  jmp #.wait

x res 1
```



打包数据模式

打包数据模式	s 用于有效地传送字节数据类型，方法是让主机端将其从接收到的字节、字或长整型数据中解包。同样，字节可以在字内发送，
多头和单词 c	并在很长一段时间内发送以提高效率。

要建立打包数据操作，您必须指定下面列出的模式之一，后跟可选的“ALT”和“SIGNED”关键字：

打包模式 {ALT} {签名}

ALT 关键字将导致发送的每个字节内的位、双位或半字节在主机端的每个字节内重新排序。这简化了您发送的原始数据的位字段相对于您正在使用的调试显示无序的情况。对于以标准格式组成的位图数据来说，这很可能是需要的。

The SIGNED 关键字将导致所有解包数据值在主机上进行符号扩展 边。

Packed-Data Modes	Descriptions	Final Values	Final Values if SIGNED
LONGS_1BIT	Each value received is translated into 32 separate 1-bit values, starting from the LSB of the received value.	0..1	-1..0
LONGS_2BIT	Each value received is translated into 16 separate 2-bit values, starting from the LSBs of the received value.	0..3	-2..1
LONGS_4BIT	Each value received is translated into 8 separate 4-bit values, starting from the LSBs of the received value.	0..15	-8..7
LONGS_8BIT	Each value received is translated into 4 separate 8-bit values, starting from the LSBs of the received value.	0..255	-128..127
LONGS_16BIT	Each value received is translated into 2 separate 16-bit values, starting from the LSBs of the received value.	0..65,535	-32,768..32,767
WORDS_1BIT	Each value received is translated into 16 separate 1-bit values, starting from the LSB of the received value.	0..1	-1..0
WORDS_2BIT	Each value received is translated into 8 separate 2-bit values, starting from the LSBs of the received value.	0..3	-2..1
WORDS_4BIT	Each value received is translated into 4 separate 4-bit values, starting from the LSBs of the received value.	0..15	-8..7
WORDS_8BIT	Each value received is translated into 2 separate 8-bit values, starting from the LSBs of the received value.	0..255	-128..127
BYTES_1BIT	Each value received is translated into 8 separate 1-bit values, starting from the LSB of the received value.	0..1	-1..0
BYTES_2BIT	Each value received is translated into 4 separate 2-bit values, starting from the LSBs of the received value.	0..3	-2..1
BYTES_4BIT	Each value received is translated into 2 separate 4-bit values, starting from the LSBs of the received value.	0..15	-8..7

用于智能引脚配置的内置符号

Smart Pin Symbol Value	Symbol Name	Details
A Input Polarity	(pick one)	
%0000_0000_000_000000000000_00_00000_0	P_TRUE_A (default)	True A input
%1000_0000_000_000000000000_00_00000_0	P_INVERT_A	Invert A input
A Input Selection	(pick one)	
%0000_0000_000_000000000000_00_00000_0	P_LOCAL_A (default)	Select local pin for A input
%0001_0000_000_000000000000_00_00000_0	P_PLUS1_A	Select pin+1 for A input
%0010_0000_000_000000000000_00_00000_0	P_PLUS2_A	Select pin+2 for A input
%0011_0000_000_000000000000_00_00000_0	P_PLUS3_A	Select pin+3 for A input
%0100_0000_000_000000000000_00_00000_0	P_OUTBIT_A	Select OUT bit for A input
%0101_0000_000_000000000000_00_00000_0	P_MINUS3_A	Select pin-3 for A input
%0110_0000_000_000000000000_00_00000_0	P_MINUS2_A	Select pin-2 for A input
%0111_0000_000_000000000000_00_00000_0	P_MINUS1_A	Select pin-1 for A input
B Input Polarity	(pick one)	
%0000_0000_000_000000000000_00_00000_0	P_TRUE_B (default)	True B input
%0000_1000_000_000000000000_00_00000_0	P_INVERT_B	Invert B input
B Input Selection	(pick one)	
%0000_0000_000_000000000000_00_00000_0	P_LOCAL_B (default)	Select local pin for B input
%0000_0001_000_000000000000_00_00000_0	P_PLUS1_B	Select pin+1 for B input
%0000_0010_000_000000000000_00_00000_0	P_PLUS2_B	Select pin+2 for B input
%0000_0011_000_000000000000_00_00000_0	P_PLUS3_B	Select pin+3 for B input
%0000_0100_000_000000000000_00_00000_0	P_OUTBIT_B	Select OUT bit for B input
%0000_0101_000_000000000000_00_00000_0	P_MINUS3_B	Select pin-3 for B input
%0000_0110_000_000000000000_00_00000_0	P_MINUS2_B	Select pin-2 for B input
%0000_0111_000_000000000000_00_00000_0	P_MINUS1_B	Select pin-1 for B input
A, B Input Logic	(pick one)	
%0000_0000_000_000000000000_00_00000_0	P_PASS_AB (default)	Select A, B
%0000_0000_001_000000000000_00_00000_0	P_AND_AB	Select A & B, B
%0000_0000_010_000000000000_00_00000_0	P_OR_AB	Select A B, B
%0000_0000_011_000000000000_00_00000_0	P_XOR_AB	Select A ^ B, B
%0000_0000_100_000000000000_00_00000_0	P_FILT0_AB	Select FILT0 settings for A, B
%0000_0000_101_000000000000_00_00000_0	P_FILT1_AB	Select FILT1 settings for A, B
%0000_0000_110_000000000000_00_00000_0	P_FILT2_AB	Select FILT2 settings for A, B
%0000_0000_111_000000000000_00_00000_0	P_FILT3_AB	Select FILT3 settings for A, B
Low-Level Pin Modes	(pick one)	
Logic/Schmitt/Comparator Input Modes		
%0000_0000_000_000000000000_00_00000_0	P_LOGIC_A (default)	Logic level A → IN, output OUT
%0000_0000_000_000100000000_00_00000_0	P_LOGIC_A_FB	Logic level A → IN, output feedback
%0000_0000_000_001000000000_00_00000_0	P_LOGIC_B_FB	Logic level B → IN, output feedback
%0000_0000_000_001100000000_00_00000_0	P_SCHMITT_A	Schmitt trigger A → IN, output OUT
%0000_0000_000_010000000000_00_00000_0	P_SCHMITT_A_FB	Schmitt trigger A → IN, output feedback
%0000_0000_000_010100000000_00_00000_0	P_SCHMITT_B_FB	Schmitt trigger B → IN, output feedback
%0000_0000_000_011000000000_00_00000_0	P_COMPARE_AB	A > B → IN, output OUT
%0000_0000_000_011100000000_00_00000_0	P_COMPARE_AB_FB	A > B → IN, output feedback
%xxxx_xxxx_xxx_xxxxSIOHHLLL_xx_xxxxx_x		Sync mode, IN/output polarity, high/low drive
ADC Input Modes		
%0000_0000_000_100000000000_00_00000_0	P_ADC_GIO	ADC GIO → IN, output OUT
%0000_0000_000_100001000000_00_00000_0	P_ADC_VIO	ADC VIO → IN, output OUT

%0000_0000_000_1000100000000_00_00000_0	P_ADC_FLOAT	ADC FLOAT → IN, output OUT
%0000_0000_000_1000110000000_00_00000_0	P_ADC_1X	ADC 1x → IN, output OUT
%0000_0000_000_1001000000000_00_00000_0	P_ADC_3X	ADC 3.16x → IN, output OUT
%0000_0000_000_1001010000000_00_00000_0	P_ADC_10X	ADC 10x → IN, output OUT
%0000_0000_000_1001100000000_00_00000_0	P_ADC_30X	ADC 31.6x → IN, output OUT
%0000_0000_000_1001110000000_00_00000_0	P_ADC_100X	ADC 100x → IN, output OUT
%xxxx_xxxx_xxx_xxxxxx0HHHLLL_xx_xxxxx_x		O = output polarity, HHH/LLL = high/low drive
DAC Output Modes		DIR enables output, OUT enables ADC
%0000_0000_000_1010000000000_00_00000_0	P_DAC_990R_3V	DAC 990Ω, 3.3V peak, ADC 1x → IN
%0000_0000_000_1010100000000_00_00000_0	P_DAC_600R_2V	DAC 600Ω, 2.0V peak, ADC 1x → IN
%0000_0000_000_1011000000000_00_00000_0	P_DAC_124R_3V	DAC 123.75Ω, 3.3V peak, ADC 1x → IN
%0000_0000_000_1011100000000_00_00000_0	P_DAC_75R_2V	DAC 75Ω, 2.0V peak, ADC 1x → IN
%xxxx_xxxx_xxx_xxxxxDDDDDDDD_xx_xxxxx_x		DDDDDDDD = 8-bit DAC value
Level-Comparison Modes		DIR enables output (1.5kΩ drive)
%0000_0000_000_1100000000000_00_00000_0	P_LEVEL_A	A > Level → IN, output OUT
%0000_0000_000_1101000000000_00_00000_0	P_LEVEL_A_FBN	A > Level → IN, output negative feedback
%0000_0000_000_1110000000000_00_00000_0	P_LEVEL_B_FBP	B > Level → IN, output positive feedback
%0000_0000_000_1111000000000_00_00000_0	P_LEVEL_B_FBN	B > Level → IN, output negative feedback
%xxxx_xxxx_xxx_xxxxSLLLLLLLL_xx_xxxxx_x		S = Synchronous, LLLLLLLL = 8-bit Level
Low-Level Pin Sub-Modes		
Sync Mode	(pick one)	(for Logic/Schmitt/Comparator/Level modes)
%xxxx_xxxx_xxx_xxxxSxxxxxxxx_xx_xxxxx_x		Sync mode bit
%0000_0000_000_0000000000000_00_00000_0	P_ASYNC_IO (default)	Select asynchronous I/O
%0000_0000_000_0000100000000_00_00000_0	P_SYNC_IO	Select synchronous I/O
IN Polarity	(pick one)	(for Logic/Schmitt/Comparator modes)
%xxxx_xxxx_xxx_xxxxIxxxxxxx_xx_xxxxx_x		IN polarity bit
%0000_0000_000_0000000000000_00_00000_0	P_TRUE_IN (default)	True IN bit
%0000_0000_000_0000010000000_00_00000_0	P_INVERT_IN	Invert IN bit
Output Polarity	(pick one)	(for Logic/Schmitt/Comparator/ADC modes)
%xxxx_xxxx_xxx_xxxxxx0xxxxxx_xx_xxxxx_x		Output polarity bit
%0000_0000_000_0000000000000_00_00000_0	P_TRUE_OUTPUT (default)	Select true output
%0000_0000_000_0000001000000_00_00000_0	P_INVERT_OUTPUT	Select inverted output
Drive-High Strength	(pick one)	(for Logic/Schmitt/Comparator/ADC modes)
%xxxx_xxxx_xxx_xxxxxxxHHHxxx_xx_xxxxx_x		Drive-high selector bits
%0000_0000_000_0000000000000_00_00000_0	P_HIGH_FAST (default)	Drive high fast (30mA)
%0000_0000_000_0000000001000_00_00000_0	P_HIGH_1K5	Drive high 1.5kΩ
%0000_0000_000_0000000010000_00_00000_0	P_HIGH_15K	Drive high 15kΩ
%0000_0000_000_0000000011000_00_00000_0	P_HIGH_150K	Drive high 150kΩ
%0000_0000_000_0000000100000_00_00000_0	P_HIGH_1mA	Drive high 1mA
%0000_0000_000_0000000101000_00_00000_0	P_HIGH_100UA	Drive high 100μA
%0000_0000_000_0000000110000_00_00000_0	P_HIGH_10UA	Drive high 10μA
%0000_0000_000_0000000111000_00_00000_0	P_HIGH_FLOAT	Float high
Drive-Low Strength	(pick one)	(for Logic/Schmitt/Comparator/ADC modes)
%xxxx_xxxx_xxx_xxxxxxxxxxLLL_xx_xxxxx_x		Drive-low selector bits
%0000_0000_000_0000000000000_00_00000_0	P_LOW_FAST (default)	Drive low fast (30mA)
%0000_0000_000_0000000000001_00_00000_0	P_LOW_1K5	Drive low 1.5kΩ
%0000_0000_000_0000000000010_00_00000_0	P_LOW_15K	Drive low 15kΩ
%0000_0000_000_0000000000011_00_00000_0	P_LOW_150K	Drive low 150kΩ
%0000_0000_000_000000000100_00_00000_0	P_LOW_1mA	Drive low 1mA
%0000_0000_000_000000000101_00_00000_0	P_LOW_100UA	Drive low 100μA
%0000_0000_000_000000000110_00_00000_0	P_LOW_10UA	Drive low 10μA

%0000_0000_000_0000000000111_00_00000_0	P_LOW_FLOAT	Float low
DIR/OUT Control	(pick one)	
%0000_0000_000_000000000000_00_00000_0	P_TT_00 (default)	TT = %00
%0000_0000_000_000000000000_01_00000_0	P_TT_01	TT = %01
%0000_0000_000_000000000000_10_00000_0	P_TT_10	TT = %10
%0000_0000_000_000000000000_11_00000_0	P_TT_11	TT = %11
%0000_0000_000_000000000000_01_00000_0	P_OE	Enable output in smart pin mode
%0000_0000_000_000000000000_01_00000_0	P_CHANNEL	Enable DAC channel in non-smart pin DAC mode
%0000_0000_000_000000000000_10_00000_0	P_BITDAC	Enable BITDAC for non-smart pin DAC mode
Smart Pin Modes	(pick one)	
%0000_0000_000_000000000000_00_00000_0	P_NORMAL (default)	Normal mode (not smart pin mode)
%0000_0000_000_000000000000_00_00001_0	P_REPOSITORY	Long repository (non-DAC mode)
%0000_0000_000_000000000000_00_00001_0	P_DAC_NOISE	DAC Noise (DAC mode)
%0000_0000_000_000000000000_00_00010_0	P_DAC_DITHER_RND	DAC 16-bit random dither (DAC mode)
%0000_0000_000_000000000000_00_00011_0	P_DAC_DITHER_PWM	DAC 16-bit PWM dither (DAC mode)
%0000_0000_000_000000000000_00_00100_0	P_PULSE	Pulse/cycle output
%0000_0000_000_000000000000_00_00101_0	P_TRANSITION	Transition output
%0000_0000_000_000000000000_00_00110_0	P_NCO_FREQ	NCO frequency output
%0000_0000_000_000000000000_00_00111_0	P_NCO_DUTY	NCO duty output
%0000_0000_000_000000000000_00_01000_0	P_PWM_TRIANGLE	PWM triangle output
%0000_0000_000_000000000000_00_01001_0	P_PWM_SAWTOOTH	PWM sawtooth output
%0000_0000_000_000000000000_00_01010_0	P_PWM_SMPs	PWM switch-mode power supply I/O
%0000_0000_000_000000000000_00_01011_0	P_QUADRATURE	A-B quadrature encoder input
%0000_0000_000_000000000000_00_01100_0	P_REG_UP	Inc on A-rise when B-high
%0000_0000_000_000000000000_00_01101_0	P_REG_UP_DOWN	Inc on A-rise when B-high, dec on A-rise when B-low
%0000_0000_000_000000000000_00_01110_0	P_COUNT_RISES	Inc on A-rise, optionally dec on B-rise
%0000_0000_000_000000000000_00_01111_0	P_COUNT_HIGHS	Inc on A-high, optionally dec on B-high
%0000_0000_000_000000000000_00_10000_0	P_STATE_TICKS	For A-low and A-high states, count ticks
%0000_0000_000_000000000000_00_10001_0	P_HIGH_TICKS	For A-high states, count ticks
%0000_0000_000_000000000000_00_10010_0	P_EVENTS_TICKS	For X A-highs/rises/edges, count ticks / Timeout on X ticks of no A-high/rise/edge
%0000_0000_000_000000000000_00_10011_0	P_PERIODS_TICKS	For X periods of A, count ticks
%0000_0000_000_000000000000_00_10100_0	P_PERIODS_HIGHS	For X periods of A, count highs
%0000_0000_000_000000000000_00_10101_0	P_COUNTER_TICKS	For periods of A in X+ ticks, count ticks
%0000_0000_000_000000000000_00_10110_0	P_COUNTER_HIGHS	For periods of A in X+ ticks, count highs
%0000_0000_000_000000000000_00_10111_0	P_COUNTER_PERIODS	For periods of A in X+ ticks, count periods
%0000_0000_000_000000000000_00_11000_0	P_ADC	ADC sample/filter/capture, internally clocked
%0000_0000_000_000000000000_00_11001_0	P_ADC_EXT	ADC sample/filter/capture, externally clocked
%0000_0000_000_000000000000_00_11010_0	P_ADC_SCOPE	ADC scope with trigger
%0000_0000_000_000000000000_00_11011_0	P_USB_PAIR	USB pin pair
%0000_0000_000_000000000000_00_11100_0	P_SYNC_TX	Synchronous serial transmit
%0000_0000_000_000000000000_00_11101_0	P_SYNC_RX	Synchronous serial receive
%0000_0000_000_000000000000_00_11110_0	P_ASYNC_TX	Asynchronous serial transmit
%0000_0000_000_000000000000_00_11111_0	P_ASYNC_RX	Asynchronous serial receive

流光模式的内置符号

Streamer Symbol Value	Symbol Name
#Immediate → LUT → Pins / DACs	
%0000_0000_0000_0000 << 16 %0000_DDDD_EPPP_BBBB << 16	X_IMM_32X1_LUT
%0001_0000_0000_0000 << 16 %0001_DDDD_EPPP_BBBB << 16	X_IMM_16X2_LUT
%0010_0000_0000_0000 << 16 %0010_DDDD_EPPP_BBBB << 16	X_IMM_8X4_LUT
%0011_0000_0000_0000 << 16 %0011_DDDD_EPPP_BBBB << 16	X_IMM_4X8_LUT
#Immediate → Pins / DACs	
%0100_0000_0000_0000 << 16 %0100_DDDD_EPPP_PPPA << 16	X_IMM_32X1_1DAC1
%0101_0000_0000_0000 << 16 %0101_DDDD_EPPP_PP0A << 16	X_IMM_16X2_2DAC1
%0101_0000_0000_0010 << 16 %0101_DDDD_EPPP_PP1A << 16	X_IMM_16X2_1DAC2
%0110_0000_0000_0000 << 16 %0110_DDDD_EPPP_P00A << 16	X_IMM_8X4_4DAC1
%0110_0000_0000_0010 << 16 %0110_DDDD_EPPP_P01A << 16	X_IMM_8X4_2DAC2
%0110_0000_0000_0100 << 16 %0110_DDDD_EPPP_P10A << 16	X_IMM_8X4_1DAC4
%0110_0000_0000_0110 << 16 %0110_DDDD_EPPP_0110 << 16	X_IMM_4X8_4DAC2
%0110_0000_0000_0111 << 16 %0110_DDDD_EPPP_0111 << 16	X_IMM_4X8_2DAC4
%0110_0000_0000_1110 << 16 %0110_DDDD_EPPP_1110 << 16	X_IMM_4X8_1DAC8
%0110_0000_0000_1111 << 16 %0110_DDDD_EPPP_1111 << 16	X_IMM_2X16_4DAC4
%0111_0000_0000_0000 << 16 %0111_DDDD_EPPP_0000 << 16	X_IMM_2X16_2DAC8
%0111_0000_0000_0001 << 16 %0111_DDDD_EPPP_0001 << 16	X_IMM_1X32_4DAC8
RDFAST → LUT → Pins / DACs	
%0111_0000_0000_0010 << 16 %0111_DDDD_EPPP_001A << 16	X_RFLONG_32X1_LUT
%0111_0000_0000_0100 << 16 %0111_DDDD_EPPP_010A << 16	X_RFLONG_16X2_LUT
%0111_0000_0000_0110 << 16 %0111_DDDD_EPPP_011A << 16	X_RFLONG_8X4_LUT
%0111_0000_0000_1000 << 16 %0111_DDDD_EPPP_1000 << 16	X_RFLONG_4X8_LUT
RDFAST → Pins / DACs	
%1000_0000_0000_0000 << 16 %1000_DDDD_EPPP_PPPA << 16	X_RFBYTE_1P_1DAC1
%1001_0000_0000_0000 << 16 %1001_DDDD_EPPP_PP0A << 16	X_RFBYTE_2P_2DAC1
%1001_0000_0000_0010 << 16 %1001_DDDD_EPPP_PP1A << 16	X_RFBYTE_2P_1DAC2
%1010_0000_0000_0000 << 16 %1010_DDDD_EPPP_P00A << 16	X_RFBYTE_4P_4DAC1
%1010_0000_0000_0010 << 16 %1010_DDDD_EPPP_P01A << 16	X_RFBYTE_4P_2DAC2
%1010_0000_0000_0100 << 16 %1010_DDDD_EPPP_P10A << 16	X_RFBYTE_4P_1DAC4
%1010_0000_0000_0110 << 16 %1010_DDDD_EPPP_0110 << 16	X_RFBYTE_8P_4DAC2
%1010_0000_0000_0111 << 16 %1010_DDDD_EPPP_0111 << 16	X_RFBYTE_8P_2DAC4
%1010_0000_0000_1110 << 16 %1010_DDDD_EPPP_1110 << 16	X_RFBYTE_8P_1DAC8
%1010_0000_0000_1111 << 16	X_RFWORD_16P_4DAC4

%1010_DDDD_EPPP_1111 << 16	
%1011_0000_0000_0000 << 16 %1011_DDDD_EPPP_0000 << 16	X_RFWORD_16P_2DAC8
%1011_0000_0000_0001 << 16 %1011_DDDD_EPPP_0001 << 16	X_RFLONG_32P_4DAC8
RDFAST → RGB → Pins / DACs	
%1011_0000_0000_0010 << 16 %1011_DDDD_EPPP_0010 << 16	X_RFBYTE_LUMA8
%1011_0000_0000_0011 << 16 %1011_DDDD_EPPP_0011 << 16	X_RFBYTE_RGBI8
%1011_0000_0000_0100 << 16 %1011_DDDD_EPPP_0100 << 16	X_RFBYTE_RGB8
%1011_0000_0000_0101 << 16 %1011_DDDD_EPPP_0101 << 16	X_RFWORD_RGB16
%1011_0000_0000_0110 << 16 %1011_DDDD_EPPP_0110 << 16	X_RFLONG_RGB24
Pins → DACs / WRFAST	
%1100_0000_0000_0000 << 16 %1100_DDDD_WPPP_PPPA << 16	X_1P_1DAC1_WFBYTE
%1101_0000_0000_0000 << 16 %1101_DDDD_WPPP_PP0A << 16	X_2P_2DAC1_WFBYTE
%1101_0000_0000_0010 << 16 %1101_DDDD_WPPP_PP1A << 16	X_2P_1DAC2_WFBYTE
%1110_0000_0000_0000 << 16 %1110_DDDD_WPPP_P00A << 16	X_4P_4DAC1_WFBYTE
%1110_0000_0000_0010 << 16 %1110_DDDD_WPPP_P01A << 16	X_4P_2DAC2_WFBYTE
%1110_0000_0000_0100 << 16 %1110_DDDD_WPPP_P10A << 16	X_4P_1DAC4_WFBYTE
%1110_0000_0000_0110 << 16 %1110_DDDD_WPPP_0110 << 16	X_8P_4DAC2_WFBYTE
%1110_0000_0000_0111 << 16 %1110_DDDD_WPPP_0111 << 16	X_8P_2DAC4_WFBYTE
%1110_0000_0000_1110 << 16 %1110_DDDD_WPPP_1110 << 16	X_8P_1DAC8_WFBYTE
%1110_0000_0000_1111 << 16 %1110_DDDD_WPPP_1111 << 16	X_16P_4DAC4_WFWORD
%1111_0000_0000_0000 << 16 %1111_DDDD_WPPP_0000 << 16	X_16P_2DAC8_WFWORD
%1111_0000_0000_0001 << 16 %1111_DDDD_WPPP_0001 << 16	X_32P_4DAC8_WFLONG
ADCs / Pins → DACs / WRFAST	
%1111_0000_0000_0010 << 16 %1111_DDDD_W000_0010 << 16	X_1ADC8_0P_1DAC8_WFBYTE
%1111_0000_0000_0011 << 16 %1111_DDDD_WPPP_0011 << 16	X_1ADC8_8P_2DAC8_WFWORD
%1111_0000_0000_0100 << 16 %1111_DDDD_W000_0100 << 16	X_2ADC8_0P_2DAC8_WFWORD
%1111_0000_0000_0101 << 16 %1111_DDDD_WPPP_0101 << 16	X_2ADC8_16P_4DAC8_WFLONG
%1111_0000_0000_0110 << 16 %1111_DDDD_W000_0110 << 16	X_4ADC8_0P_4DAC8_WFLONG
DDS / Goertzel	
%1111_0000_0000_0111 << 16 %1111_DDDD_0PPP_P111 << 16	X_DDS_GOERTZEL_SINC1
%1111_0000_1000_0111 << 16 %1111_DDDD_1PPP_P111 << 16	X_DDS_GOERTZEL_SINC2
Sub-Fields	
DAC Channel Outputs	
%xxxx_DDDD_xxxx_xxxx << 16 %0000_0000_0000_0000 << 16 %0000_0001_0000_0000 << 16 %0000_0010_0000_0000 << 16 %0000_0011_0000_0000 << 16 %0000_0100_0000_0000 << 16 %0000_0101_0000_0000 << 16 %0000_0110_0000_0000 << 16 %0000_0111_0000_0000 << 16 %0000_1000_0000_0000 << 16 %0000_1001_0000_0000 << 16	X_DACS_OFF (default) X_DACS_0_0_0_0 X_DACS_X_X_0_0 X_DACS_0_0_X_X X_DACS_X_X_X_0 X_DACS_X_X_0_X X_DACS_X_0_X_X X_DACS_0_X_X_X X_DACS_0N0_0N0 X_DACS_X_X_0N0

%0000_1010_0000_0000 << 16 %0000_1011_0000_0000 << 16 %0000_1100_0000_0000 << 16 %0000_1101_0000_0000 << 16 %0000_1110_0000_0000 << 16 %0000_1111_0000_0000 << 16	X_DACS_0N0_X_X X_DACS_1_0_1_0 X_DACS_X_X_1_0 X_DACS_1_0_X_X X_DACS_1N1_0N0 X_DACS_3_2_1_0
Pin Output Control	
%xxxx_xxxx_Exxx_xxxx << 16 %0000_0000_0000_0000 << 16 %0000_0000_1000_0000 << 16	X_PINS_OFF (default) X_PINS_ON
Write Control	
%xxxx_xxxx_Wxxx_xxxx << 16 %0000_0000_0000_0000 << 16 %0000_0000_1000_0000 << 16	X_WRITE_OFF (default) X_WRITE_ON
Alternate Order for 1/2/4 bits	
%xxxx_xxxx_xxxx_xxxA << 16 %0000_0000_0000_0000 << 16 %0000_0000_0000_0001 << 16	X_ALT_OFF (default) X_ALT_ON

事件和中断源的内置符号

Symbol Value	Symbol Name
0	EVENT_INT / INT_OFF
1	EVENT_CT1
2	EVENT_CT2
3	EVENT_CT3
4	EVENT_SE1
5	EVENT_SE2
6	EVENT_SE3
7	EVENT_SE4
8	EVENT_PAT
9	EVENT_FBW
10	EVENT_XMT
11	EVENT_XFI
12	EVENT_XRO
13	EVENT_XRL
14	EVENT_ATN
15	EVENT_QMT

COGINIT 用法的内置符号

COGINIT Symbol Value	Symbol Name	Details
%00_0000	COGEXEC (default)	Use "COGEXEC + CogNumber" to start a cog in cogexec mode
%10_0000	HUBEXEC	Use "HUBEXEC + CogNumber" to start a cog in hubexec mode
%01_0000	COGEXEC_NEW	Starts an available cog in cogexec mode
%11_0000	HUBEXEC_NEW	Starts an available cog in hubexec mode
%01_0001	COGEXEC_NEW_PAIR	Starts an available eve/odd pair of cogs in cogexec mode, useful for LUT sharing
%11_0001	HUBEXEC_NEW_PAIR	Starts an available eve/odd pair of cogs in hubexec mode, useful for LUT sharing

COGSPIN 用法的内置符号

COGINIT Symbol Value	Symbol Name	Details
%01_0000	NEWCOG	Starts an available cog

内置数字符号

Symbol Value	Symbol Name	Details
\$0000_0000	FALSE	Same as 0
\$FFFF_FFFF	TRUE	Same as -1
\$8000_0000	NEGX	Negative-extreme integer, -2_147_483_648 (\$8000_0000)
\$7FFF_FFFF	POSX	Positive-extreme integer, +2_147_483_647 (\$7FFF_FFFF)
\$4049_0FDB	PI	Single-precision floating-point value of Pi, 3.14159265

PNut.exe 的命令行选项

Command	Action	ERROR.TXT file afterwards (file will contain one of these lines)
pnut	Start PNut.exe.	okay
pnut filename	Load <i>filename</i> (.spin2 extension is assumed, but not enforced).	okay
pnut filename -c	Load and compile <i>filename</i> , then exit.	okay <filename_path>:<line_number>:error:<error_message>
pnut filename -r	Load, compile, and run <i>filename</i> , then exit.	okay <filename_path>:<line_number>:error:<error_message> serial_error
pnut filename -rd	Load, compile, run, and debug <i>filename</i> , then exit when the Debug window gets closed.	okay <filename_path>:<line_number>:error:<error_message> serial_error
pnut filename -f	Load, compile, and program flash <i>filename</i> , then exit.	okay <filename_path>:<line_number>:error:<error_message> serial_error
pnut filename -fd	Load, compile, program flash, and debug <i>filename</i> , then exit when the Debug window gets closed.	okay <filename_path>:<line_number>:error:<error_message> serial_error
pnut -debug {CommPort} {BaudRate}	Open CommPort (default = 1) at BaudRate (default = 2_000_000) and present incoming DEBUG data and displays.	okay serial_error

在包含批处理文件以调用 PNut.exe 并将状态返回到 STDOUT、STDERR 和 ERRORLEVEL

PNUT_SHELL.BAT File	Batch File Line Descriptions
@echo off set ERROR_FILE=error.txt if exist %ERROR_FILE% del /q /f %ERROR_FILE% if exist %1 set GOOD_SRC=1 if exist %1.spin2 set GOOD_SRC=1 if defined GOOD_SRC (pnut_v35L %1 %2 %3 set pnuterror = %ERRORLEVEL% for /f "tokens=" %%i in (%ERROR_FILE%) do echo %%i 1>&2) else (set pnuterror=-1 echo "Error: File NOT found - %1" 1>&2) exit %pnuterror%	Cancel echo to console. Set ERROR.TXT filename. If ERROR.TXT exists, delete it. Check first parameter for a valid source file. Check first parameter for a valid .spin2 source file. IF source file exists ...Invoke PNut with passed parameters. Example: pnut_shell filename -r ...Capture ERRORLEVEL from PNut (0 = okay, 1 = error). ...Copy ERROR.TXT file to STDOUT and STDERR. ELSE ...Set file-not-found error. ...Return file-not-found error message to STDOUT and STDERR. Return ERRORLEVEL. Change to 'exit /b %pnuterror%' to maintain the console window.

时钟设置

到埃斯塔完成程序的初始时钟设置后，您可以声明编译器将查找的某些符号来确定您的设置。这些符号必须以一种方式定义
跟随 克组合：

的

CON declarations (numbers are for example, can vary)	Effect	HUBSET %CC_SS **
CON _clkfreq = 250_000_000 _errfreq = 0	Selects XI/XO-crystal-plus-PLL mode, assumes 20MHz crystal. The optimal PLL setting will be computed to achieve _clkfreq. Compilation fails if _clkfreq ± _errfreq is unachievable. *	10_11
CON _xtlfreq = 12_000_000 _clkfreq = 148_500_000 _errfreq = 150_000	Selects XI/XO-crystal-plus-PLL mode, along with frequencies. The optimal PLL setting will be computed to achieve _clkfreq. Compilation fails if _clkfreq ± _errfreq is unachievable. *	1x_11
CON _xinfreq = 32_000_000 _clkfreq = 297_500_000 _errfreq = 100_000	Selects XI-input-plus-PLL mode, along with frequencies. The optimal PLL setting will be computed to achieve _clkfreq. Compilation fails if _clkfreq ± _errfreq is unachievable. *	01_10
CON _xtlfreq = 16_000_000	Selects XI/XO-crystal mode and frequency.	1x_10
CON _xinfreq = 100_000_000	Selects XI-input mode and frequency.	01_10
CON _rcslow	Selects internal RCSLOW oscillator which runs at ~20KHz.	00_01
CON _rcfast	Selects internal RCFAST oscillator which runs at 20MHz+. This is the default mode, in case nothing is specified.	00_00

* _errfreq 声明是可选的，因为 _errfreq 默认为 1_000_000。 ** 如果 _xtlfreq >= 16_000_000，则
x=0 表示每个 XI/XO 15pF，否则 x=1 表示每个 XI/XO 30pF。

在编译期间，编译器定义了两个常量符号，其值反映了编译后的时钟设置：

Symbol	Description
clkmode_	The compiled clock mode, settable via HUBSET. - For Spin2 programs, HUBSET will be invoked with 'clkmode_' before your program starts, in order to set the compiled clock mode. The 'clkmode_' value will also be stored in the hub variable 'clkmode'. - For pure PASM programs, 'clkmode_' can be used to set the clock mode away from its initial RCFAST setting to any crystal/PLL compiled setting, as follows:<pre>HUBSET ##clkmode_ & !3 'start crystal/PLL, stay in RCFAST WAITX ##20_000_000/100 'wait 10ms HUBSET ##clkmode_ 'switch to crystal/PLL</pre> - The 'clkmode_' value may differ in each file of the application hierarchy. Files below the top-level file do not inherit the top-level file's value.
clkfreq_	The compiled clock frequency. - For Spin2 programs, the 'clkfreq_' value will be stored in the hub variable 'clkfreq'. - For pure PASM programs, 'clkfreq_' may be referenced only as a constant. - The 'clkfreq_' value may differ in each file of the application hierarchy. Files below the top-level file do not inherit the top-level file's value.

对于 Spin2 程序，维护两个反映当前时钟设置的集线器变量：

Spin2 Variables	Description
clkmode	The current clock mode, located at LONG[\$40]. Initialized with the 'clkmode_' value.
clkfreq	The current clock frequency, located at LONG[\$44]. Initialized with the 'clkfreq_' value.
	- For Spin2 methods, these variables can be read and written as 'clkmode' and 'clkfreq'. Rather than write these variables directly, it's much safer to use: <pre>CLKSET(new_clkmode, new_clkfreq)</pre> This way, all other code sees a quick, parallel update to both 'clkmode' and 'clkfreq', and the clock mode transition is done safely, employing the prior values, in order to avoid a potential clock glitch. - For PASM code running under Spin2, these variables can be read and written as follows:<pre>RDLONG x, #@clkmode 'read clkmode into x WRLONG x, #@clkmode 'write x to clkmode RDLONG x, #@clkfreq 'read clkfreq into x WRLONG x, #@clkfreq 'write x to clkfreq SETQ #2-1 RDLONG x, #@clkmode 'read clkmode and clkfreq into x and x+1 SETQ #2-1 WRLONG x, #@clkmode 'write x and x+1 to clkmode and clkfreq</pre>

对于仅限 PASM 的程序，有一条名为 ASMCLK 的特殊指令，它将设置由时钟设置符号指定的时钟模式。ASMCLK 没有操作数，但可以与条件前缀一起使用。ASMCLK 将汇编为 1 条或 6 条 P ASM 指令，具体取决于时钟模式。该指令应在仅 PASM 程序启动时使用一次，假设当前选择从启动继承的 RCFast 模式：

CON declarations (numbers are for example, can vary)	HUBSET %CC_SS	ASMCLK assembles to:
CON _clkfreq = 250_000_000 _errfreq = 0	10_11	<pre>HUBSET ##clkmode_ & !%11 'start external clock, stay in RCFast mode WAITX ##20_000_000/100 'allow 10ms for external clock to stabilize HUBSET ##clkmode_ 'switch to external clock mode</pre>
CON _xtlfreq = 12_000_000 _clkfreq = 148_500_000 _errfreq = 150_000	1x_11	
CON _xinfreq = 32_000_000 _clkfreq = 297_500_000 _errfreq = 100_000	01_10	
CON _xtlfreq = 16_000_000	1x_10	
CON _xinfreq = 100_000_000	01_10	
CON _rcslow	00_01	<pre>HUBSET #1 'switch to RCLow mode</pre>
CON _rcfast	00_00	<pre>HUBSET #0 'stay in RCFast mode</pre>